LENGUAJES DE PROGRAMACIÓN I

Universidad Nacional Mayor de San Marcos · Facultad de Ingeniería Electrónica y Eléctrica

# Números reales en memoria y valor de una resistencia por colores (Java)

## Teoría desde cero, programas en Java comentados y probados, diagramas y guion completo para la exposición

**Parte 1:** ¿Cómo se almacena un número real en la memoria? (IEEE 754)

**Parte 2:** Programa que recibe los nombres de los colores y calcula el valor de la resistencia

**Lenguaje:** Java (NetBeans, proyectos Maven)

**Incluye:** Guion de exposición para compañeros que no saben Java

- Java explicado desde C++
- Bits, bytes y memoria
- Binario y notación científica
- Float y double (IEEE 754)
- Sesgo, mantisa y redondeo
- Little-endian
- Código de colores
- Programa original transcrito
- Programa nuevo: colores a valor
- Pruebas y versión mejorada
- Diagramas de flujo
- Cómo exponer: guion y preguntas

Material de estudio no oficial. Los programas fueron compilados y ejecutados; las salidas mostradas son reales. El programa de la resistencia se verificó con sus 1 200 combinaciones de colores.

# Índice


**Cómo usar esta guía** 2

- Archivos de esta carpeta 2
- Orden de lectura recomendado 2
- Sobre el código de las imágenes 3
- Todo fue probado 3

**Java desde cero (partiendo de lo que ya saben de C++)** 4

- 1. ¿Qué es Java? 4
- 2. NetBeans y los proyectos Maven 5
- 3. Estructura de un programa en Java 6
- 4. Tabla de equivalencias C++ ↔ Java 6
- 5. Tipos de datos 7
- 6. Leer datos con `Scanner` 8
- 7. Mostrar datos 8
- 8. Decisiones y repeticiones (igual que en C++) 8
- 9. Textos ( `String` ) en Java 9
- 10. Métodos (funciones) 9
- 11. Mini-resumen para la exposición 10

**Parte 1 — ¿Cómo se almacena un número real en la memoria?** 11

- 1. La pregunta (y una sorpresa) 11
- 2. Bit, byte y memoria 11
- 3. El sistema binario 11
- 4. Convertir un decimal a binario (a mano) 12
- 5. ¿Cómo se guardan los enteros? (para comparar) 13
- 6. La idea: notación científica en binario («punto flotante») 13
- 7. El estándar IEEE 754 14
- 8. Ejemplo completo: −6.75 como `float` 15
- 9. Ejemplo con trampa: 0.1 como `float` 16
- 10. Valores especiales 16
- 11. ¿En qué orden quedan los bytes en la RAM? 17
- 12. Consecuencias prácticas 17
- 13. El programa Java: `Numero_Real_en_Memoria` 18
- 14. Resumen en 8 ideas 23
- 15. Preguntas para comprobar lo aprendido 24

**Parte 2 — El valor de una resistencia a partir de sus colores** 25

- 1. ¿Qué es una resistencia? 25
- 2. ¿Por qué tienen bandas de colores? 26
- 3. Cómo leer una resistencia de 4 bandas 27
- 4. El programa de clase (el de las imágenes) 28
- 5. Nuestro programa: colores → valor 30
- 6. Versión mejorada (opcional) 39
- 7. Cómo ejecutar los programas en NetBeans 40

**Guion de exposición** 42

- 1. Objetivo y público 42

2. Estructura y tiempos ..... 42
3. Antes del día de la exposición ..... 42
4. Diapositivas sugeridas ..... 43
5. Guion: qué decir en cada bloque ..... 44
6. Cómo explicar Java a quien no sabe Java ..... 47
7. Datos de prueba (para tener impresos) ..... 47
8. Errores que conviene evitar al exponer ..... 48
9. Preguntas frecuentes (y cómo responderlas) ..... 49
10. Plan B si la demostración falla ..... 49
11. Tarjeta de bolsillo ..... 50

# Cómo usar esta guía

Esta carpeta resuelve la tarea de las **imágenes del código en NetBeans**, que tiene **dos partes**:

| Parte | Pregunta | Qué se entrega |
|---|---|---|
| **1** | ¿Cómo se almacena un **número real** en la memoria de la computadora? | Teoría desde cero (binario, notación científica, estándar IEEE 754), diagramas y un **programa Java** que muestra los bits reales de cualquier número |
| **2** | Rehacer el programa de las imágenes para que, **en vez de ingresar números, se ingresen los nombres de los colores** y el programa dé el **valor de la resistencia** | Teoría de resistencias y código de colores, el **programa Java** nuevo, explicación línea por línea, pruebas y una versión mejorada opcional |
| **Exposición** | ¿Cómo explicarlo a compañeros que no saben Java? | **Guion de exposición** con estructura, tiempos, qué decir, demostración y preguntas frecuentes |

## Archivos de esta carpeta

| Archivo o carpeta | Contenido |
|---|---|
| `Guia_Java_Numero_Real_y_Resistencia.pdf` | Esta guía completa en PDF (para leer o imprimir) |
| `Guion_de_Exposicion.pdf` | Solo el guion de la exposición, para tenerlo a mano el día de la presentación |
| `Parte1_Numero_Real_en_Memoria/` | Proyecto de NetBeans (Maven) del programa de números reales |
| `Parte2_Valor_Resistencia_segun_Colores/` | Proyecto de NetBeans (Maven) del programa de la resistencia (versión principal + versión mejorada) |
| `imagenes_originales/` | Las capturas del código de clase y su **transcripción completa** en un archivo `.java` |
| `diagramas/` | Todas las figuras de la guía |
| `guia/` | El texto de esta guía (se lee directamente en GitHub) |

## Orden de lectura recomendado

1. **Java desde cero**: lo mínimo de Java para entender ambos programas, explicado a partir de lo que ya saben de C++.
2. **Parte 1 — El número real en memoria**.

3. **Parte 2 — La resistencia por colores**.
4. **Guion de exposición**: cómo presentarlo.

Si van a exponer en grupo, todos deberían leer el capítulo «Java desde cero» y luego cada uno profundizar en la parte que va a presentar.

## Sobre el código de las imágenes

- Las capturas mostraban las líneas 1–50 y 67–98. Las líneas **51–64** no se veían: se reconstruyeron siguiendo exactamente el mismo patrón del programa (y coinciden con la numeración de las capturas). Lo que faltaba al final (la «sexta imagen») es:

```
            if(n==0)
                opc = 1;
        }
        System.out.print("\n****** FIN *******");
    }
}
```

- Los recuadros grises que se ven en NetBeans, como `s:`, `source:`, `a:` y `b:`, **no son parte del código**: NetBeans los dibuja para indicar el nombre de cada parámetro. No se escriben (ver el capítulo «Java desde cero»). Por eso, la última línea correcta es simplemente `System.out.print("\n****** FIN *******");`.
- La transcripción completa está en `imagenes_originales/Colores_bandas_segun_valor_Resistencia_Varios2.java`; compila y funciona igual que el original.

## Todo fue probado

- Ambos programas se compilaron con Maven (como lo hace NetBeans) y también para Java 8, sin advertencias.
- Las ejecuciones que aparecen en la guía son **salidas reales**.
- El programa de la resistencia se verificó automáticamente con las **1 200 combinaciones** posibles de colores (10 × 10 × 12): todas dan el valor correcto. La versión mejorada se verificó con sus 10 800 combinaciones.

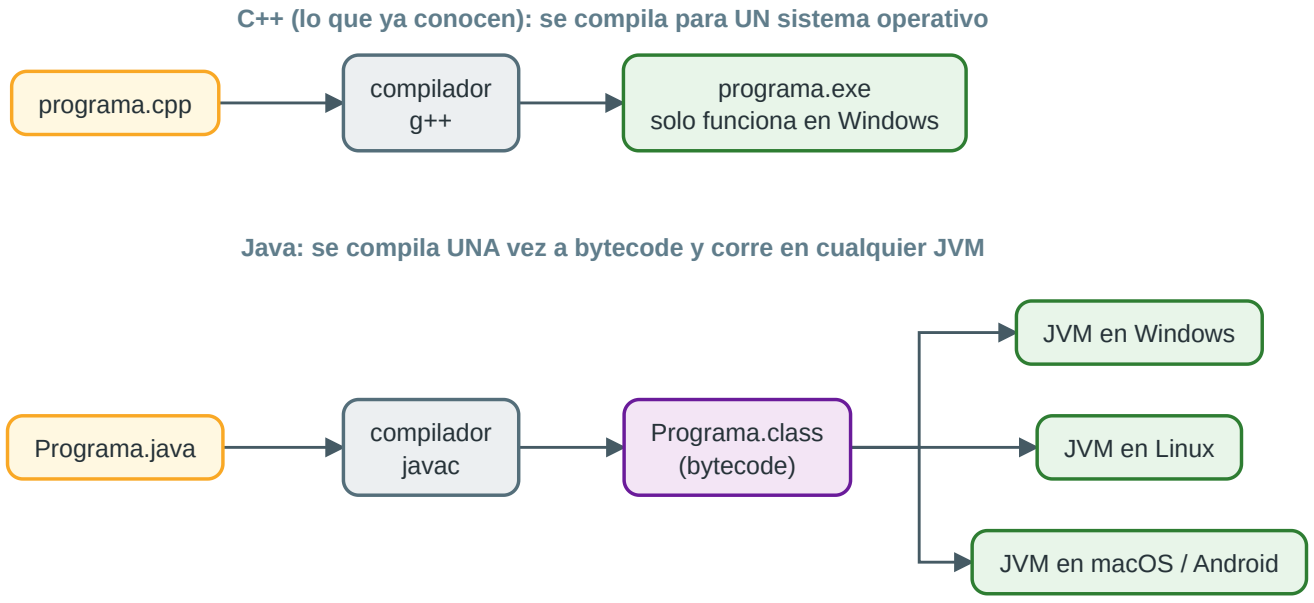
# Java desde cero (partiendo de lo que ya saben de C++)

Este capítulo explica **solo lo necesario** para entender los dos programas. La idea central para la exposición es: **Java se parece muchísimo a C++**; cambian algunos nombres, pero la lógica (variables, `if`, `switch`, `while`, operadores) es la misma.

## 1. ¿Qué es Java?

**Java** es un lenguaje de programación de alto nivel y orientado a objetos, creado en 1995 por James Gosling en *Sun Microsystems* (hoy pertenece a Oracle). Se usa en aplicaciones de escritorio, servidores, sistemas bancarios y en Android.

Su lema es *«escríbelo una vez, ejecútalo en cualquier lugar»*. La diferencia clave con C++ está en **cómo se traduce el programa**:




*C++ se compila para un sistema; Java se compila a bytecode que ejecuta la JVM*

- En **C++**, el compilador produce un `.exe` en lenguaje máquina **para un sistema operativo** concreto.
- En **Java**, el compilador `javac` produce un archivo `.class` con **bytecode**, un código intermedio. Luego, la **Máquina Virtual de Java (JVM)** lo ejecuta. Como hay una JVM para Windows, otra para Linux, otra para macOS, etc., **el mismo `.class` funciona en todas**.

| Sigla | Nombre | Para qué sirve |
|---|---|---|
| **JVM** | *Java Virtual Machine* | Ejecuta el bytecode |
| **JRE** | *Java Runtime Environment* | JVM + bibliotecas: lo necesario para **ejecutar** programas Java |
| **JDK** | *Java Development Kit* | JRE + compilador ( `javac` ) y herramientas: lo necesario para **programar** |

## 2. NetBeans y los proyectos Maven

**NetBeans** es un **IDE** (*entorno de desarrollo integrado*): un editor que además compila, ejecuta y ayuda a encontrar errores. Los proyectos del curso son de tipo **Maven** (lo indica la ventana *Output* del profesor: `Building … 1.0-SNAPSHOT`, `maven-resources-plugin`).

Estructura de un proyecto Maven:

```
Valor_Resistencia_segun_Colores_Bandas/       <- carpeta del proyecto
├── pom.xml                                   <- configuración del proyecto (Maven)
└── src/main/java/
    └── com/mycompany/valor_resistencia_segun_colores_bandas/
        └── Valor_Resistencia_segun_Colores_Bandas.java   <- el programa
```

- `com.mycompany.…` es el **paquete** (*package*): una carpeta que agrupa clases. NetBeans lo crea a partir del nombre del proyecto.
- El nombre del archivo **debe ser igual** al nombre de la clase pública (`Valor_Resistencia_segun_Colores_Bandas.java` ↔ `public class Valor_Resistencia_segun_Colores_Bandas`).

Atajos útiles:

| Acción | Atajo |
|---|---|
| Ejecutar el proyecto | `F6` |
| Ejecutar solo el archivo abierto | `Shift` + `F6` |
| Abrir un proyecto existente | *File → Open Project* (`Ctrl` + `Shift` + `O`) |
| Crear un proyecto | *File → New Project* (`Ctrl` + `Shift` + `N`) |
| Ordenar (formatear) el código | `Alt` + `Shift` + `F` |

Cuando el programa pide datos, **se escriben en la ventana *Output*** (abajo) y se presiona Enter.

### ¿Qué son «s:», «source:», «a:» y «b:»?

En las capturas del profesor aparecen cosas como `System.out.print(s: "...")` o `Math.pow(a: 10, b: ncb3)`. **Eso no es código**: son **pistas de parámetros** (*inline parameter hints*) que NetBeans dibuja en gris para indicar el nombre de cada parámetro del método. **No se escriben**; si se escribieran, el programa no compilaría.

**¿Qué son «s:», «source:», «a:» y «b:» que aparecen en NetBeans?**

```
Scanner datos = new Scanner( source: System.in);
```

```
vr = vr * Math.pow( a: 10,  b: ncb3);
```

```
System.out.print( s: "\n****** FIN *******");
```

**Los recuadros grises los DIBUJA NetBeans: son los NOMBRES de los parámetros del método.**
NO forman parte del código y NO se escriben. Si copias el código, escribe solo lo que está en negrita:
**`System.out.print("\n****** FIN *******");`**

*Las pistas grises de NetBeans no forman parte del código*

## 3. Estructura de un programa en Java

```java
package com.mycompany.saludo;           // el paquete (carpeta) del archivo

public class Saludo                     // en Java TODO va dentro de una clase
{
    public static void main(String[] args)   // aquí empieza el programa
    {
        System.out.println("Hola, mundo");    // muestra el texto y salta de línea
    }
}
```

`public static void main(String[] args)` asusta un poco, pero cada palabra tiene un sentido:

| Palabra | Significado |
|---|---|
| `public` | Se puede usar desde fuera de la clase (la JVM necesita llamarlo) |
| `static` | Pertenece a la clase; no hace falta crear un objeto para ejecutarlo |
| `void` | No devuelve ningún valor (en C++, `main` devuelve `int`) |
| `main` | El nombre obligatorio del punto de inicio, igual que en C++ |
| `String[] args` | Una lista de textos que se le puede pasar al programa al iniciarlo (no la usamos) |

## 4. Tabla de equivalencias C++ ↔ Java

Esta tabla es **la herramienta principal para explicar Java** a quien ya vio C++:

| Concepto | C++ | Java |
|---|---|---|
| Usar una biblioteca | `#include <iostream>` | `import java.util.Scanner;` |
| Programa principal | `int main() { … return 0; }` | `public static void main(String[] args) { … }` |
| Mostrar y saltar línea | `cout << "Hola" << endl;` | `System.out.println("Hola");` |
| Mostrar sin saltar | `cout << "Hola";` | `System.out.print("Hola");` |
| Unir texto y variables | `cout << "Valor: " << vr;` | `System.out.print("Valor: " + vr);` |
| Preparar la lectura | (no hace falta) | `Scanner datos = new Scanner(System.in);` |
| Leer un entero | `cin >> n;` | `n = datos.nextInt();` |
| Leer una palabra | `cin >> s;` | `s = datos.next();` |
| Leer una línea | `getline(cin, s);` | `s = datos.nextLine();` |
| Texto | `string` | `String` (con **S** mayúscula) |
| Lógico | `bool` | `boolean` |
| Comparar dos textos | `s1 == s2` | `s1.equals(s2)` (**no** `==`) |
| Potencia | `pow(10, 3)` con `<cmath>` | `Math.pow(10, 3)` |
| Subcadena | `s.substr(inicio, cantidad)` | `s.substring(inicio, fin)` (¡posición **final**, no cantidad!) |
| Longitud | `s.length()` | `s.length()` |
| Convertir a real | `static_cast<double>(a)` | `(double) a` |
| `if`, `else`, `switch`, `while`, `for` | Igual | Igual |
| Operadores `+ - * / %`, `&& \|\| !`, `== !=` | Igual | Igual (incluida la división entera: `7 / 2` da `3`) |

## 5. Tipos de datos

En Java el tamaño de cada tipo es **el mismo en todas las computadoras**:

| Tipo | Guarda | Tamaño | Rango / precisión |
|---|---|---|---|
| `byte` | Entero pequeño | 1 byte | −128 a 127 |
| `short` | Entero | 2 bytes | −32 768 a 32 767 |
| `int` | Entero | 4 bytes | ≈ ±2 147 millones |
| `long` | Entero grande | 8 bytes | ≈ $\pm 9.2 \times 10^{18}$ |
| `float` | Real (simple precisión) | 4 bytes | ~7 cifras significativas |
| `double` | Real (doble precisión) | 8 bytes | ~15–16 cifras significativas |
| `char` | Un carácter | 2 bytes | Letras de casi todos los idiomas (Unicode) |
| `boolean` | Lógico | — | `true` o `false` |
| `String` | Texto | Variable | — |

El profesor usa `byte opc = 0;` para la variable que controla el ciclo: basta con 1 byte porque solo guarda 0 o 1.

## 6. Leer datos con `Scanner`

```java
import java.util.Scanner;                    // al inicio del archivo
...
Scanner datos = new Scanner(System.in);      // se crea UNA vez: "datos" leerá del teclado
String color = datos.next();                 // lee una palabra
byte n = datos.nextByte();                   // lee un número entero pequeño
double x = datos.nextDouble();               // lee un número real
String linea = datos.nextLine();             // lee una línea completa
```

- `new Scanner(System.in)` crea un **objeto** lector conectado al teclado ( `System.in` ). Es el equivalente de tener `cin` listo para usar.
- Si el usuario escribe letras donde se espera un número ( `nextByte()` , `nextInt()` ), el programa **se detiene con un error** ( `InputMismatchException` ). Por eso, en nuestro programa, la respuesta a «¿Desea terminar?» se lee como texto con `next()` .
- `nextDouble()` usa la configuración regional de la computadora: en algunas configuraciones en español hay que escribir `6,75` en vez de `6.75` . Por eso el programa de la Parte 1 lee el número como texto y cambia la coma por punto.

## 7. Mostrar datos

- `System.out.print(x)` muestra sin saltar de línea; `System.out.println(x)` muestra y salta.
- `"\n"` dentro de un texto también hace un salto de línea (como en C++). El profesor escribe `System.out.print("\n...")` .
- El operador `+` **une textos**: `"El valor es: " + vr + " Ohm"` .

> **Trampa de examen**
>
> Cuidado con mezclar sumas y textos: `"Total: " + 2 + 3` muestra **`Total: 23`** (une el 2 y luego el 3), mientras que `"Total: " + (2 + 3)` muestra **`Total: 5`** . Los paréntesis obligan a sumar primero.

Un detalle de Java: los `double` muy grandes o muy pequeños se muestran en **notación científica**: `47000000.0` aparece como `4.7E7` , que significa $4.7 \times 10^7$. Lo veremos en el programa de la resistencia.

## 8. Decisiones y repeticiones (igual que en C++)

**`if` / `else` :**

```java
if(sw==0)
{
    // si todos los colores fueron válidos
}
else
    System.out.print("No se puede calcular");
```

**`switch`:** elige un camino según el valor de una variable. Cada `case` es un valor posible; `break` sale del `switch`; `default` se ejecuta si ningún `case` coincidió.

```
switch(colorb1)                 // desde Java 7 se puede usar switch con String
{
    case "negro":  ncb1 = 0;
                   break;       // sin break, seguiría ejecutando el case siguiente
    case "marron": ncb1 = 1;
                   break;
    default:       sw = 1;      // ningún case coincidió: color inválido
}
```

**`while`:** repite mientras la condición sea verdadera.

```
byte opc = 0;
while(opc==0)          // se repite mientras opc valga 0
{
    // ... pedir colores, calcular, mostrar ...
    // si el usuario escribe 0:  opc = 1;  -> el ciclo termina
}
```

## 9. Textos ( `String` ) en Java

| Método | Qué hace | Ejemplo | Resultado |
|---|---|---|---|
| `s.length()` | Cantidad de caracteres | `"FIEE-2026-0145".length()` | `14` |
| `s.substring(i, f)` | Desde `i` **hasta antes de** `f` | `"FIEE-2026-0145".substring(5, 9)` | `"2026"` |
| `s.substring(i)` | Desde `i` hasta el final | `"FIEE-2026-0145".substring(10)` | `"0145"` |
| `s.toLowerCase()` | Todo a minúsculas | `"ROJO".toLowerCase()` | `"rojo"` |
| `s.equals(t)` | ¿Son iguales los textos? | `"rojo".equals("rojo")` | `true` |
| `s.replace(a, b)` | Cambia `a` por `b` | `"6,75".replace(',', '.')` | `"6.75"` |

**Importante**

En Java, **los textos se comparan con `equals`**, nunca con `==`. El operador `==` compara si son *el mismo objeto en memoria*, no si tienen las mismas letras: dos textos `"rojo"` creados por separado pueden dar `false` con `==` y `true` con `equals`.

## 10. Métodos (funciones)

Un **método** es un bloque de código con nombre que se puede llamar varias veces, como una función de C++. El programa de la Parte 1 y la versión mejorada de la Parte 2 los usan:

```
static int numeroDelColor(String color)   // recibe un texto y devuelve un int
{
    switch(color)
    {
        case "negro": return 0;
        case "marron": return 1;
        // ...
        default: return -1;               // -1 significa "color no válido"
    }
}
```

- `static`: se puede llamar directamente desde `main` sin crear objetos.
- `int`: el tipo del valor que devuelve (`void` si no devuelve nada).
- `return`: entrega el resultado y termina el método.

## 11. Mini-resumen para la exposición

1. Java = C++ con otros nombres: `System.out.print` es `cout`, `Scanner` es `cin`, `String` es `string`.
2. Todo va dentro de una **clase**, y el programa empieza en **`main`**.
3. `if`, `switch`, `while` y los operadores son **iguales** a C++.
4. Los textos se comparan con **`equals`**.
5. Lo gris que dibuja NetBeans (`s:`, `a:`, `b:`) **no se escribe**.

# Parte 1 — ¿Cómo se almacena un número real en la memoria?

## 1. La pregunta (y una sorpresa)

La computadora solo guarda **ceros y unos**. Guardar un entero como 19 es fácil, pero ¿cómo guarda un número con decimales como **6.75**, **0.1** o **4.7**?

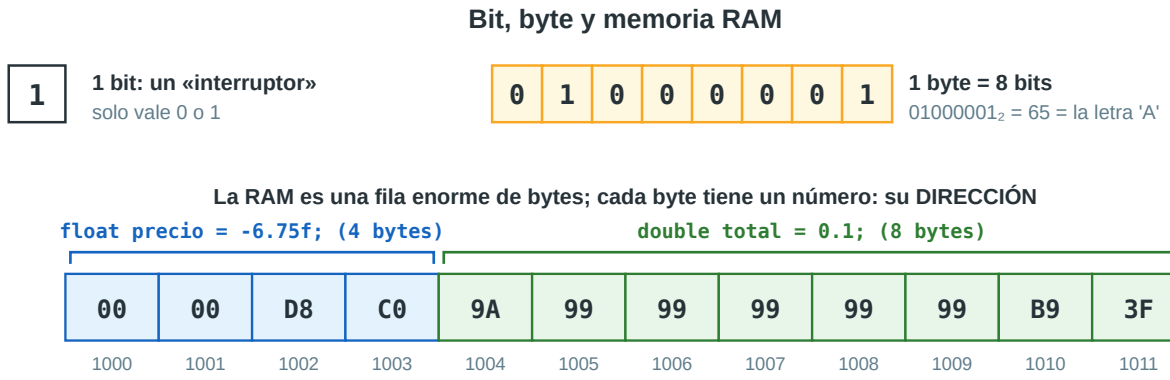
Antes de responder, una sorpresa que sirve para abrir la exposición. En Java (y en C++, Python, Excel…):

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 ?  false
```

Al terminar este capítulo se entenderá **exactamente por qué** ocurre esto.

## 2. Bit, byte y memoria

- Un **bit** es la unidad mínima de información: vale **0 o 1** (como un interruptor apagado o encendido).
- Un **byte** son **8 bits**. Con un byte se pueden formar $2^8 = 256$ combinaciones (por ejemplo, la letra `A` es `01000001`, es decir, 65).
- La **memoria RAM** es como una fila enorme de casilleros de 1 byte. Cada casillero tiene un número único, su **dirección**.
- Una variable ocupa **varios bytes seguidos**: un `float` ocupa 4 bytes y un `double`, 8.




*Bit, byte y RAM: un float ocupa 4 bytes y un double 8 bytes*

## 3. El sistema binario

En el sistema **decimal** (base 10) cada posición vale 10 veces más que la de su derecha. Lo mismo pasa con los decimales, pero dividiendo:

> $6.75 = 6 \times 10^{0} + 7 \times 10^{-1} + 5 \times 10^{-2} = 6 + 0.7 + 0.05$

En **binario** (base 2) solo hay dos dígitos (0 y 1) y cada posición vale **el doble** de la de su derecha:

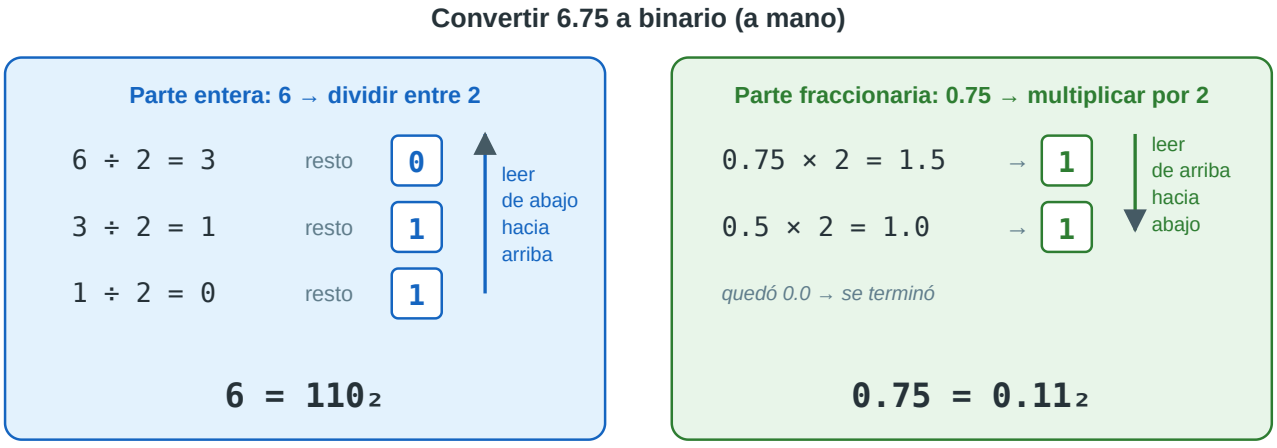
| Posición | $2^3$ | $2^2$ | $2^1$ | $2^0$ | . | $2^{-1}$ | $2^{-2}$ | $2^{-3}$ | $2^{-4}$ |
|---|---|---|---|---|---|---|---|---|---|
| Valor | 8 | 4 | 2 | 1 | . | 0.5 | 0.25 | 0.125 | 0.0625 |
| $110.11_2$ | | 1 | 1 | 0 | . | 1 | 1 | | |

> $110.11_2 = 4 + 2 + 0 + 0.5 + 0.25 = \mathbf{6.75}$

## 4. Convertir un decimal a binario (a mano)

Se trabaja por separado la **parte entera** y la **parte fraccionaria**:

- **Parte entera**: se divide entre 2 repetidamente y se anotan los **restos**; el número binario son los restos **leídos de abajo hacia arriba**.
- **Parte fraccionaria**: se multiplica por 2 repetidamente y se anota la **parte entera** de cada resultado (0 o 1); el binario son esos dígitos **leídos de arriba hacia abajo**. Se sigue con la parte decimal que queda, hasta que llegue a 0.




*Conversión de 6.75 a binario paso a paso*

### El problema: algunos decimales son infinitos en binario

Hagamos lo mismo con **0.1**:

| Operación | Resultado | Bit |
|---|---|---|
| 0.1 × 2 | 0.2 | 0 |
| 0.2 × 2 | 0.4 | 0 |
| 0.4 × 2 | 0.8 | 0 |
| 0.8 × 2 | **1**.6 | 1 |
| 0.6 × 2 | **1**.2 | 1 |
| 0.2 × 2 | 0.4 | 0 ← ¡ya habíamos visto 0.2! |
| … | … | se repite para siempre |

$0.1 = 0.0001100110011001100\ldots_2$ — **nunca termina**. Es como 1/3 en decimal (0.3333…): no se puede escribir exacto con una cantidad finita de dígitos. Como la memoria es finita, **hay que cortar y redondear**. Ese pequeño error es el origen del 0.30000000000000004.

| Número | ¿Exacto en binario? | Por qué |
|---|---|---|
| 6.75 | Sí | 0.75 = 1/2 + 1/4 |
| 0.5, 0.25, 0.125 | Sí | Son potencias de 2 |
| 4700 | Sí | Todo entero (no muy grande) es exacto |
| 0.1, 0.2, 0.3, 4.7 | **No** | Tienen infinitos dígitos binarios |

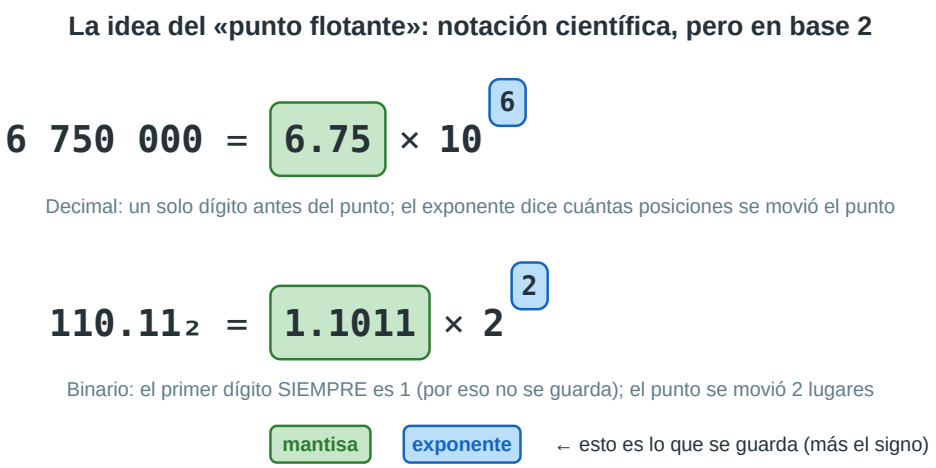
## 5. ¿Cómo se guardan los enteros? (para comparar)

Un `int` se guarda directamente en binario con 32 bits: 19 → `00000000 00000000 00000000 00010011`. Los negativos usan un método llamado **complemento a dos**. Los enteros se guardan **siempre exactos**, pero solo pueden ser enteros y tienen un rango limitado (±2 147 millones en un `int`).

Para los números reales se necesita otra idea, porque deben poder ser **muy grandes** (la distancia a una estrella) y **muy pequeños** (la carga de un electrón) a la vez.

## 6. La idea: notación científica en binario («punto flotante»)

En ciencias escribimos 6 750 000 como **$6.75 \times 10^6$**: un número corto (la **mantisa**) por una potencia de 10 (el **exponente**). La computadora hace lo mismo, pero en **base 2**:




*Notación científica decimal y binaria*

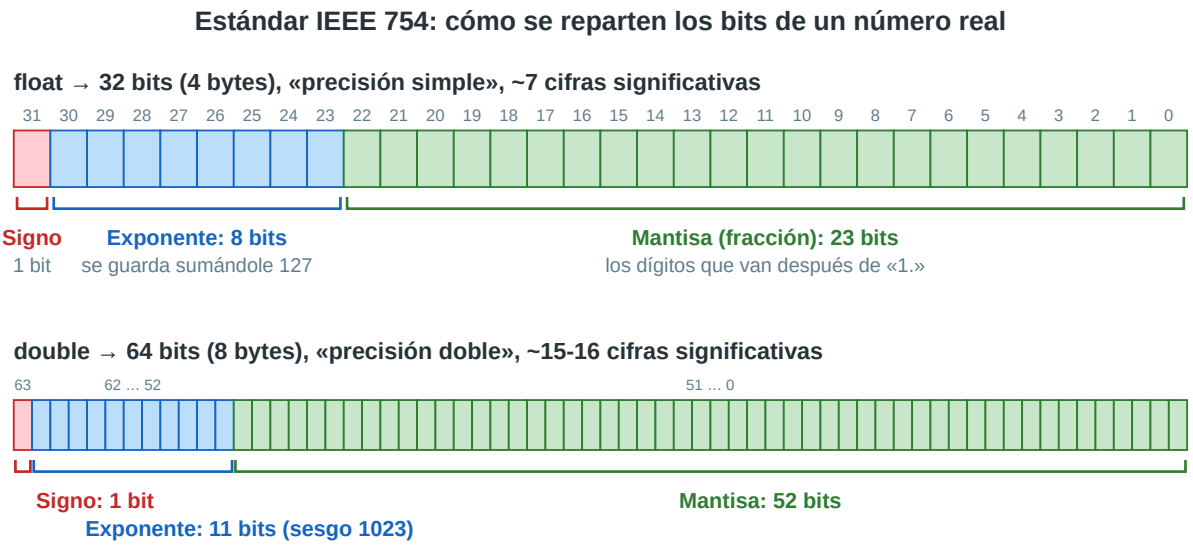
> $110.11_2 = \mathbf{1.1011}_2 \times 2^2$

Mover el punto y compensar con el exponente se llama **normalizar**. Como el punto «se mueve» (flota) según el exponente, estos números se llaman de **punto flotante** (*floating point*); de ahí viene el nombre del tipo `float`.

Detalle clave: en binario, después de normalizar, **el primer dígito siempre es 1** (el único dígito distinto de 0 que existe). Como siempre es 1, **no hace falta guardarlo**: se gana un bit de precisión «gratis». Se le llama **bit implícito**.

# 7. El estándar IEEE 754

Desde 1985, prácticamente todas las computadoras guardan los reales con el estándar **IEEE 754**. Cada número se reparte en **tres campos**:




$$\text{valor} = (-1)^{\text{signo}} \times 1.\text{mantisa}_2 \times 2^{(\text{exponente} - \text{sesgo})}$$

sesgo = 127 en float · sesgo = 1023 en double · el «1.» no se guarda (bit implícito): ¡se gana 1 bit gratis!

*Reparto de bits en float (32) y double (64)*

| | `float` (precisión simple) | `double` (precisión doble) |
|---|---|---|
| Tamaño total | 32 bits = 4 bytes | 64 bits = 8 bytes |
| Signo | 1 bit (0 = +, 1 = −) | 1 bit |
| Exponente | 8 bits, **sesgo 127** | 11 bits, **sesgo 1023** |
| Mantisa (fracción) | 23 bits (+ 1 implícito = 24) | 52 bits (+ 1 implícito = 53) |
| Cifras decimales confiables | ~7 | ~15–16 |
| Valor máximo | $\approx 3.4 \times 10^{38}$ | $\approx 1.8 \times 10^{308}$ |
| Valor positivo mínimo (normal) | $\approx 1.2 \times 10^{-38}$ | $\approx 2.2 \times 10^{-308}$ |
| En Java / C++ | `float` (literal `6.75f` ) | `double` (literal `6.75` ) |

La fórmula para reconstruir el valor es:

> **valor = (−1)^signo × 1.mantisa₂ × 2^(exponente − sesgo)**

## ¿Para qué sirve el «sesgo»?

El exponente puede ser negativo (0.1 = $1.6 \times 2^{-4}$), pero el campo de exponente se guarda como un número **sin signo**. El truco es **sumarle un sesgo** antes de guardarlo:

- En `float` , exponente real −4 → se guarda −4 + 127 = **123** → `01111011` .

- Al leerlo: 123 − 127 = −4.

Así, el campo de 8 bits (0 a 255) representa exponentes reales de −126 a +127; los valores 0 y 255 se reservan para casos especiales (sección 10).

## 8. Ejemplo completo: −6.75 como `float`

| Paso | Qué se hace | Resultado |
|---|---|---|
| 1 | **Signo**: es negativo | bit de signo = **1** |
| 2 | **Binario** de 6.75 (sin el signo) | $110.11_2$ |
| 3 | **Normalizar**: dejar un solo 1 antes del punto | $1.1011_2 \times 2^2$ |
| 4 | **Exponente con sesgo**: 2 + 127 | 129 = **$10000001_2$** |
| 5 | **Mantisa**: los bits después del «1.», completando 23 bits con ceros | **1011**00000000000000000000 |
| 6 | **Juntar**: signo · exponente · mantisa | 1 10000001 10110000000000000000000 |
| 7 | **Hexadecimal** (grupos de 4 bits) | **C0D80000** |




| | | |
|---|---|---|
| **Signo** | 1 | → el número es NEGATIVO |
| **Exponente** | $10000001_2$ = 128 + 1 = 129 | → exponente real = 129 − 127 = 2 |
| **Mantisa** | 1011000…0 | → $1.1011_2$ = 1 + 1/2 + 0/4 + 1/8 + 1/16 = 1.6875 |

**valor = $(-1)^1 \times 1.6875 \times 2^2 = -1.6875 \times 4 = -6.75$**

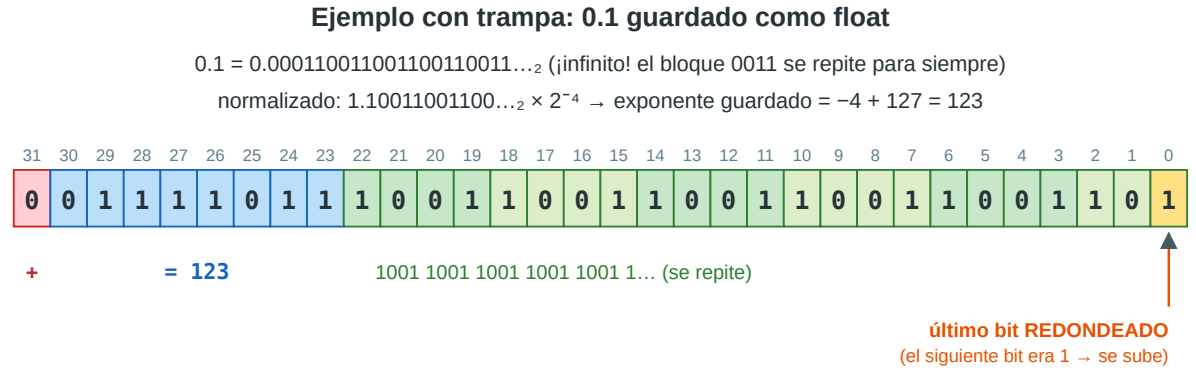
¡Exacto! 6.75 tiene un binario finito (110.11), así que se guarda sin error.

*Los 32 bits de −6.75 y cómo se leen*

**Comprobación** con la fórmula: $(-1)^1 \times 1.1011_2 \times 2^{(129 - 127)} = -1.6875 \times 4 =$ **−6.75** ✔. Como 6.75 tiene un binario finito, se guarda **sin ningún error**.

## 9. Ejemplo con trampa: 0.1 como `float`

| Paso | Resultado |
|---|---|
| Signo | positivo → **0** |
| Binario | $0.0001100110011001100\ldots_2$ (infinito) |
| Normalizar | $1.100110011001100\ldots_2 \times 2^{-4}$ |
| Exponente con sesgo | −4 + 127 = 123 → **01111011** |
| Mantisa | Se toman 23 bits: `10011001100110011001100` y el bit 24 es **1** → **se redondea hacia arriba**: `10011001100110011001101` |
| Resultado (hex) | **3DCCCCCD** |




**Lo que REALMENTE queda guardado:**

`float : 0.100000001490116119384765625`

`double: 0.1000000000000000055511151231257827…`

Ninguno es exactamente 0.1. Java imprime «0.1» porque muestra la forma más corta que lo identifica.

**Por eso en la computadora 0.1 + 0.2 da 0.30000000000000004**

*Los 32 bits de 0.1: el patrón se repite y el último bit se redondea*

Por el redondeo, el `float` no guarda 0.1 sino **0.100000001490116119384765625**, y el `double` guarda **0.1000000000000000055511151231257827…**. Java muestra «0.1» porque imprime el número más corto que identifica a ese valor, pero **por dentro no es 0.1**. Al sumar 0.1 + 0.2, los pequeños errores de ambos se acumulan y el resultado (el `double` más cercano a la suma) se imprime como 0.30000000000000004.

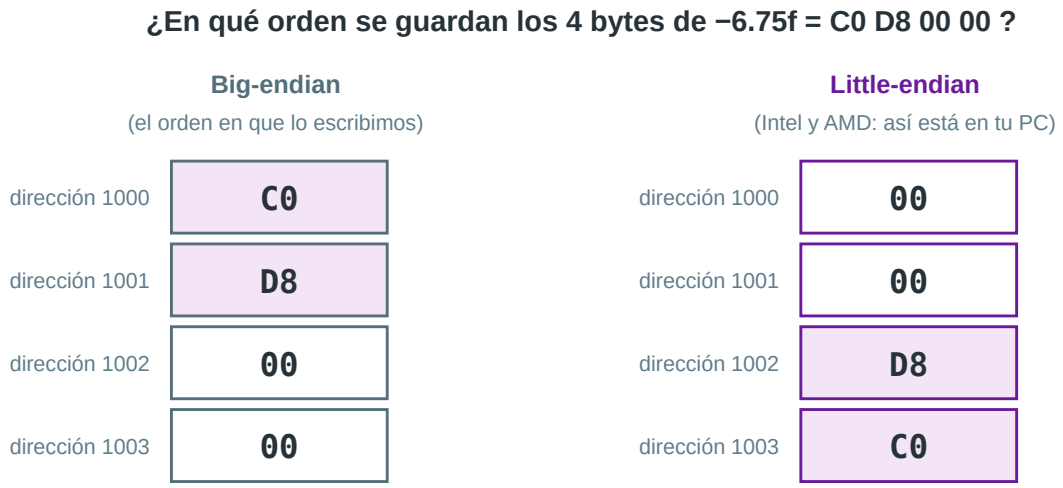
## 10. Valores especiales

Los valores extremos del campo de exponente se reservan para casos especiales:

| Exponente | Mantisa | Significado | Ejemplo en Java |
|---|---|---|---|
| Todo ceros | Cero | **Cero** (existe +0 y −0) | `0.0`, `-0.0` |
| Todo ceros | Distinta de cero | Números **subnormales** (más pequeños que el mínimo normal) | `Float.MIN_VALUE` |
| Todo unos | Cero | **Infinito** (+∞ o −∞) | `1.0 / 0` → `Infinity` |
| Todo unos | Distinta de cero | **NaN** (*Not a Number*, «no es un número») | `0.0 / 0` → `NaN` |

## 11. ¿En qué orden quedan los bytes en la RAM?

Un `float` ocupa 4 bytes seguidos, pero ¿en qué orden? En los procesadores **Intel y AMD** (y en la gran mayoría de celulares) se usa el orden **little-endian**: **el byte menos significativo va primero**, en la dirección más baja.




Little-endian = «el byte menos significativo primero». El programa de la Parte 1 muestra este orden.

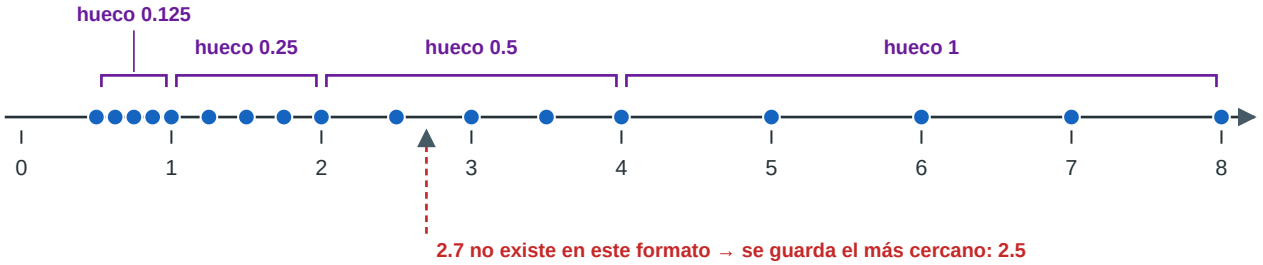
*Big-endian vs little-endian para −6.75f*

Java oculta este detalle al programador (siempre «ve» el número completo), pero así es como queda físicamente en la memoria de una PC. El programa de esta parte muestra los bytes en ese orden.

## 12. Consecuencias prácticas

1. **Precisión limitada.** Un `float` tiene ~7 cifras: 16 777 217 se guarda como 16 777 216 (no alcanzan los bits). Por eso en Java y C++ se usa `double` por defecto.
2. **No comparar reales con `==`.** En lugar de `a == b`, se compara con una tolerancia: `Math.abs(a - b) < 1e-9`.
3. **Los errores se acumulan.** Sumar 0.1 diez veces da 0.9999999999999999.
4. **Dinero.** Los sistemas de bancos y tiendas no usan `double` para el dinero: usan céntimos enteros o tipos decimales exactos (`BigDecimal` en Java).
5. **Los valores posibles no están repartidos de forma pareja**: hay muchos cerca del cero y cada vez menos (más separados) a medida que el número crece.

**Los números que SÍ se pueden guardar no están repartidos de forma pareja**




Formato «de juguete» con solo 2 bits de mantisa. El float usa 23 y el double 52 (puntos mucho más juntos), pero el patrón es el mismo: cada vez que el número se duplica, el hueco entre valores posibles también se duplica.

*Los huecos entre valores representables crecen con el número*

> **Nota**
>
> **Conexión con la Parte 2:** el valor de la resistencia se guarda en un `double`. El multiplicador dorado es ×0.1, pero 0.1 no es exacto en binario: `12 * 0.1` da **1.2000000000000002**. Por eso nuestro programa **divide entre 10** (12 / 10 = 1.2, el valor guardado más cercano a 1.2) en lugar de multiplicar por 0.1.

## 13. El programa Java: `Numero_Real_en_Memoria`

Proyecto: `Parte1_Numero_Real_en_Memoria/` · Archivo: `Numero_Real_en_Memoria.java`

### 13.1 Qué hace

Muestra un menú con dos opciones:

1. **Analizar un número real**: el usuario escribe un número (acepta `6.75` o `6,75`) y el programa
   - hace la **conversión a mano, paso a paso**, exactamente como en las secciones 4 y 8 (divisiones, multiplicaciones, normalización, sesgo, mantisa y redondeo);
   - muestra los **32 bits reales del `float`** y los **64 bits reales del `double`** que guarda Java, separados en signo | exponente | mantisa;
   - muestra el **hexadecimal**, el **orden de los bytes en la RAM** (little-endian), el **valor exacto guardado** y el **error de redondeo**;
   - compara el resultado hecho «a mano» con el de Java (deben **coincidir**).
2. **Curiosidades**: 0.1 + 0.2, sumar 0.1 diez veces, el límite del `float`, 1/3, los huecos entre números, infinito y NaN, y el ejemplo de la resistencia (12 × 0.1).

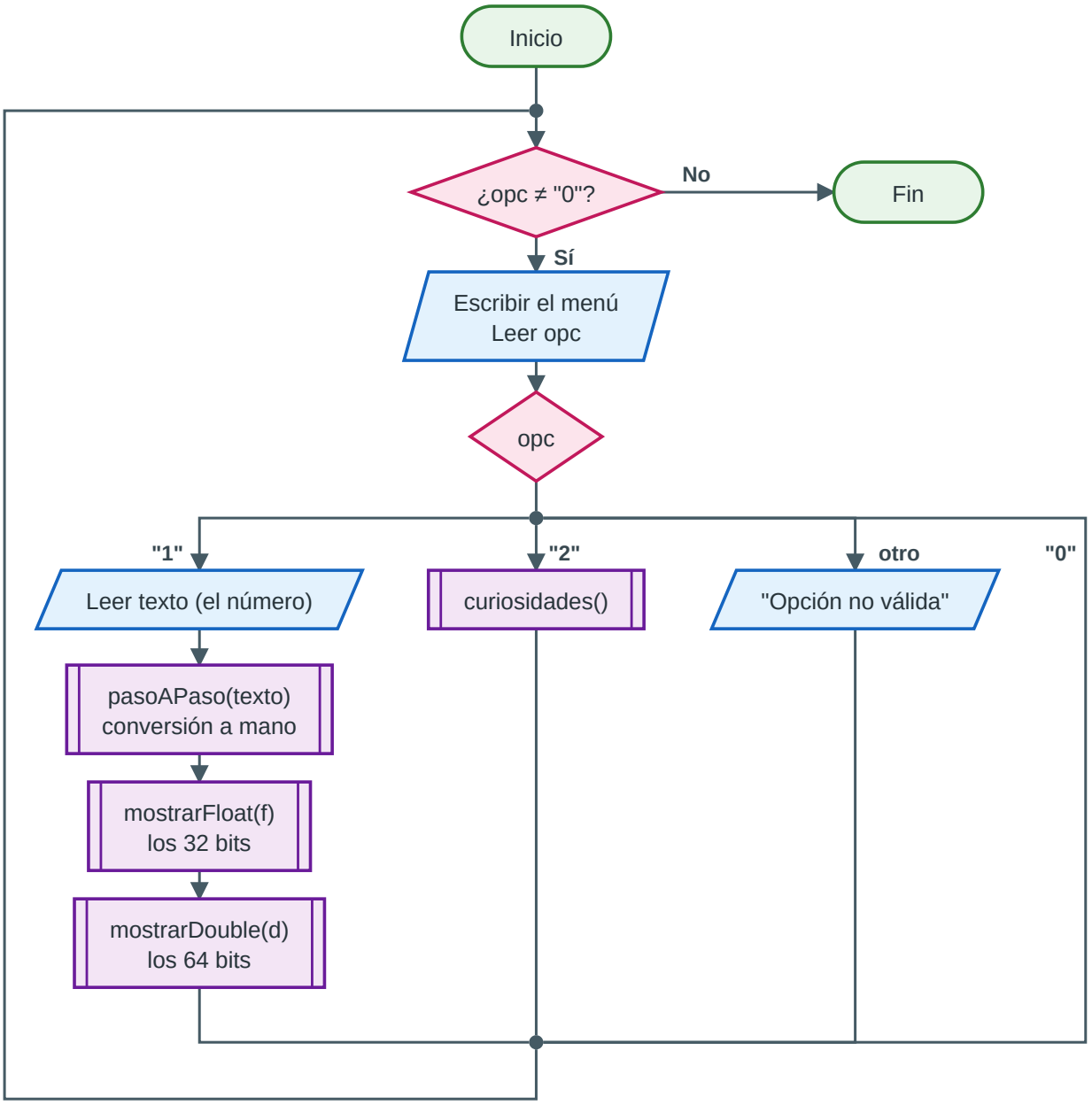



*Diagrama de flujo del programa*

## 13.2 Ejecución real con −6.75

```
Elija una opcion: 1
Ingrese un numero real (ej: -6.75   0.1   4700): -6.75

----- PASO A PASO (como en papel), formato float de 32 bits -----
1) Signo: negativo -> bit de signo = 1
2) Parte entera = 6  (dividir entre 2 y anotar el resto)
      6 / 2 = 3   resto 0
      3 / 2 = 1   resto 1
      1 / 2 = 0   resto 1
   => 6 en binario = 110  (restos leidos de abajo hacia arriba)
3) Parte fraccionaria = 0.75  (multiplicar por 2 y anotar la parte entera)
      0.75 x 2 = 1.5   -> bit 1
      0.5 x 2 = 1   -> bit 1
   => numero completo en binario = 110.11
4) Normalizar (dejar un solo 1 antes del punto):
      1.1011 x 2^2
5) Exponente guardado = 2 + 127 = 129  ->  10000001 (8 bits)
6) Mantisa: se toman 23 bits despues del punto (el "1." NO se guarda)
      10110000000000000000000 | 0 ...  <- el bit 24 decide el redondeo
      el resto se descarta (redondeo hacia abajo)
7) Resultado a mano : 1 | 10000001 | 10110000000000000000000
   Lo que guardo Java: 1 | 10000001 | 10110000000000000000000   <- COINCIDEN

----- LO QUE JAVA GUARDA: float (32 bits = 4 bytes) -----
Bits        : 1 | 10000001 | 10110000000000000000000
Signo       : 1  (negativo)
Exponente   : 10000001 = 129  ->  exponente real = 129 - 127 = 2
Mantisa     : 10110000000000000000000
Hexadecimal : C0D80000
En la RAM   : [+0]00 [+1]00 [+2]D8 [+3]C0  (little-endian)
Java lo imprime como : -6.75
Valor EXACTO guardado: -6.75
Error de redondeo    : 0  (se guardo EXACTO)

----- LO QUE JAVA GUARDA: double (64 bits = 8 bytes) -----
Bits        : 1 | 10000000001 | 1011000000000000000000000000000000000000000000000000
Signo       : 1  (negativo)
Exponente   : 10000000001 = 1025  ->  exponente real = 1025 - 1023 = 2
Mantisa     : 1011000000000000000000000000000000000000000000000000
Hexadecimal : C01B000000000000
En la RAM   : [+0]00 [+1]00 [+2]00 [+3]00 [+4]00 [+5]00 [+6]1B [+7]C0  (little-endian)
Java lo imprime como : -6.75
Valor EXACTO guardado: -6.75
Error de redondeo    : 0  (se guardo EXACTO)
```

## 13.3 Ejecución real con 0.1 (parte del `float`)

```
----- PASO A PASO (como en papel), formato float de 32 bits -----
1) Signo: positivo -> bit de signo = 0
2) Parte entera = 0  ->  en binario tambien es 0
3) Parte fraccionaria = 0.1  (multiplicar por 2 y anotar la parte entera)
      0.1 x 2 = 0.2   -> bit 0
      0.2 x 2 = 0.4   -> bit 0
      0.4 x 2 = 0.8   -> bit 0
      0.8 x 2 = 1.6   -> bit 1
      0.6 x 2 = 1.2   -> bit 1
      0.2 x 2 = 0.4   -> bit 0
      0.4 x 2 = 0.8   -> bit 0
      0.8 x 2 = 1.6   -> bit 1
      0.6 x 2 = 1.2   -> bit 1
      0.2 x 2 = 0.4   -> bit 0
      ... (54 pasos mas)
   => NUNCA llega a 0: en binario este numero tiene infinitos digitos
   => numero completo en binario = 0.0001100110011001100110011001100110011001...
4) Normalizar (dejar un solo 1 antes del punto):
      1.10011001100110011001100110011001... x 2^-4
5) Exponente guardado = -4 + 127 = 123  ->  01111011 (8 bits)
6) Mantisa: se toman 23 bits despues del punto (el "1." NO se guarda)
      10011001100110011001100 | 1 ...  <- el bit 24 decide el redondeo
      el bit 24 es 1 -> se REDONDEA hacia arriba: 10011001100110011001101
7) Resultado a mano : 0 | 01111011 | 10011001100110011001101
   Lo que guardo Java: 0 | 01111011 | 10011001100110011001101   <- COINCIDEN

----- LO QUE JAVA GUARDA: float (32 bits = 4 bytes) -----
Bits        : 0 | 01111011 | 10011001100110011001101
Signo       : 0  (positivo)
Exponente   : 01111011 = 123  ->  exponente real = 123 - 127 = -4
Mantisa     : 10011001100110011001101
Hexadecimal : 3DCCCCCD
En la RAM   : [+0]CD [+1]CC [+2]CC [+3]3D  (little-endian)
Java lo imprime como : 0.1
Valor EXACTO guardado: 0.100000001490116119384765625
Error de redondeo    : 0.000000001490116119384765625
```

## 13.4 Ejecución real de las curiosidades

```
----- CURIOSIDADES DE LOS NUMEROS REALES -----
1) 0.1 + 0.2 = 0.30000000000000004
   0.1 + 0.2 == 0.3 ?  false
   Por eso se comparan con tolerancia: Math.abs(suma - 0.3) < 1e-9 ?  true
2) Sumar 0.1 diez veces da: 0.9999999999999999  (no exactamente 1.0)
3) float 16777217 se guarda como 16777216  (el float solo tiene 24 bits de precision, ~7 cifras)
   double 16777217 se guarda como 16777217  (~16 cifras)
4) 1/3 en float  = 0.33333334
   1/3 en double = 0.3333333333333333
5) Distancia al siguiente float despues de 1.0       : 1.1920929E-7
   Distancia al siguiente float despues de 1000000.0 : 0.0625
   (mientras mas grande el numero, mas separados estan los valores posibles)
6) Valores especiales: 1.0/0 = Infinity ;  -1.0/0 = -Infinity ;  0.0/0 = NaN
7) Resistencia marron-rojo-dorado: 12 * 0.1 = 1.2000000000000002   pero 12 / 10.0 = 1.2
```

## 13.5 Las partes clave del código, explicadas

**a) Obtener los bits que Java guarda.** Java trae métodos que devuelven los bits de un real tal cual están en memoria: `Float.floatToIntBits(f)` (32 bits) y `Double.doubleToLongBits(d)` (64 bits). Luego los separamos con `substring`, igual que `substr` en C++:

```
    static void mostrarFloat(String texto, float f)
    {
        int bits = Float.floatToIntBits(f);       // Java nos da los 32 bits del float
        String b = aBinario(bits, 32);
        String s = b.substring(0, 1);             // 1 bit de signo
        String e = b.substring(1, 9);             // 8 bits de exponente
        String m = b.substring(9);                // 23 bits de mantisa
        int expGuardado = Integer.parseInt(e, 2);
        String hex = aHexadecimal(b);
```

- `aBinario(bits, 32)` convierte el número a un texto de exactamente 32 ceros y unos.
- `b.substring(1, 9)` toma los caracteres 1 al 8: los **8 bits del exponente**. (En Java, el segundo número es la posición **final**, que no se incluye.)
- `Integer.parseInt(e, 2)` convierte el texto binario `"10000001"` al número decimal **129**; luego se le resta el sesgo 127.

**b) Herramientas auxiliares.**

```
    // Escribe "valor" en binario usando exactamente nBits (rellena con ceros a la izquierda)
    static String aBinario(long valor, int nBits)
    {
        String s = Long.toBinaryString(valor);
        while(s.length() < nBits)
            s = "0" + s;
        return s.substring(s.length() - nBits);
    }

    // Convierte una cadena de bits a hexadecimal (cada 4 bits = 1 digito hex)
    static String aHexadecimal(String bits)
    {
        String hex = "";
        for(int i = 0; i < bits.length(); i += 4)
        {
            int valor = Integer.parseInt(bits.substring(i, i + 4), 2);   // 4 bits -> 0..15
            hex = hex + Integer.toHexString(valor).toUpperCase();          // 0..15 -> 0..F
        }
        return hex;
    }

    // En la RAM (procesadores Intel/AMD) los bytes se guardan del MENOS significativo al MAS
    // significativo (little-endian). Ej: 3DCCCCCD -> [+0]CD [+1]CC [+2]CC [+3]3D
    static String bytesEnMemoria(String hex)
    {
        String r = "";
        int nBytes = hex.length() / 2;
        for(int i = 0; i < nBytes; i++)
        {
            int fin = hex.length() - 2 * i;
            r = r + "[+" + i + "]" + hex.substring(fin - 2, fin) + " ";
        }
        return r.trim();
    }
```

**c) La conversión «a mano».** El método `pasoAPaso` repite el procedimiento de papel. La parte entera se divide entre 2 dentro de un `while` y cada resto se pega **a la izquierda** (así quedan «leídos de abajo hacia arriba»):

```
        long q = entero;
        while(q > 0)
        {
            System.out.print("      " + q + " / 2 = " + (q / 2)
                             + "   resto " + (q % 2) + "\n");
            binEntero = (q % 2) + binEntero;      // cada resto se escribe a la IZQUIERDA
            q = q / 2;
        }
```

La parte fraccionaria se multiplica por 2 y se guarda el bit (la parte entera del producto). Se usa `BigDecimal`, un tipo de Java que hace **cuentas decimales exactas**, para que el paso a paso sea idéntico al de papel (con `double` aparecerían los mismos errores que queremos explicar):

```
        while(fr.signum() > 0 && pasos < 64)
        {
            BigDecimal producto = fr.multiply(new BigDecimal(2));
            int bit = (producto.compareTo(BigDecimal.ONE) >= 0) ? 1 : 0;
            if(pasos < 10)                        // solo mostramos los 10 primeros pasos
                System.out.print("      " + limpio(fr) + " x 2 = " + limpio(producto)
                                 + "   -> bit " + bit + "\n");
            binFrac = binFrac + bit;
            fr = producto.subtract(new BigDecimal(bit));   // queda solo lo de despues del punto
            pasos++;
        }
```

Después, el programa normaliza, suma el sesgo, toma 23 bits de mantisa y aplica el redondeo (si el bit 24 es 1, se suma 1 a la mantisa; en un empate exacto se aplica la regla del estándar: queda el valor cuyo último bit es 0). Al final compara con los bits de Java: en todas las pruebas **coinciden**, incluso en el caso límite 16 777 217.

### 13.6 Cómo ejecutarlo

1. En NetBeans: *File* → *Open Project* y elegir la carpeta `Parte1_Numero_Real_en_Memoria`.
2. Presionar `F6` (*Run Project*). La primera vez Maven puede demorar porque descarga lo que necesita.
3. En la ventana *Output*: escribir `1`, Enter, y luego un número (por ejemplo `-6.75`, `0.1` o `4700`). Escribir `2` para las curiosidades y `0` para salir.

Números recomendados para la demostración:

| Número | Qué muestra |
|---|---|
| `-6.75` | Caso exacto: el paso a paso coincide con el ejemplo de la sección 8 |
| `0.1` | Caso infinito: patrón repetido, redondeo y error de redondeo |
| `4700` | El valor de la resistencia del ejemplo de la Parte 2 |
| `16777217` | El límite de precisión del `float` |
| `NaN` o `-0` | Valores especiales |

## 14. Resumen en 8 ideas

1. La computadora guarda todo en bits; un `float` usa 32 y un `double`, 64.
2. Los reales se guardan en **notación científica binaria**: signo, exponente y mantisa (estándar **IEEE 754**).
3. Se normaliza a la forma **1.xxxx × 2^e**; el «1.» no se guarda (bit implícito).

4. El exponente se guarda **sumándole un sesgo** (127 en `float`, 1023 en `double`).
5. Muchos decimales (0.1, 0.2, 4.7) son **infinitos en binario**: se cortan y se redondean.
6. Por eso aparecen resultados como **0.30000000000000004** y no se deben comparar reales con `==`.
7. Los valores extremos del exponente representan **0, infinito y NaN**.
8. En una PC, los bytes del número quedan en la RAM en orden **little-endian**.

## 15. Preguntas para comprobar lo aprendido

1. ¿Cuántos bits tiene cada campo de un `float`?
2. ¿Por qué el exponente se guarda con sesgo?
3. ¿Qué es el bit implícito?
4. ¿Por qué 6.75 se guarda exacto y 0.1 no?
5. ¿Qué valor hexadecimal tiene −6.75 como `float`?
6. ¿Qué significa que una PC sea little-endian?
7. ¿Por qué no conviene comparar dos `double` con `==`?

**Respuestas:** 1) Signo 1, exponente 8, mantisa 23. 2) Para poder representar exponentes negativos usando un campo sin signo. 3) El 1 que siempre aparece antes del punto al normalizar; como siempre es 1, no se guarda. 4) Porque $6.75 = 110.11_2$ tiene un binario finito, mientras que $0.1 = 0.000110011\ldots_2$ es infinito y hay que redondearlo. 5) `C0D80000`. 6) Que guarda primero (en la dirección más baja) el byte menos significativo: −6.75f queda como `00 00 D8 C0`. 7) Porque los valores pueden tener pequeños errores de redondeo: se compara con una tolerancia, `Math.abs(a - b) < 1e-9`.

# Parte 2 — El valor de una resistencia a partir de sus colores

## 1. ¿Qué es una resistencia?

Una **resistencia** (o *resistor*) es un componente electrónico que **se opone al paso de la corriente eléctrica**. Se usa, por ejemplo, para limitar la corriente que llega a un LED y que no se queme, o para repartir un voltaje.

- Su valor se mide en **ohmios** (símbolo **Ω**, se lee «ohm»).
- Se relaciona con el voltaje y la corriente por la **ley de Ohm**: **V = I × R**.
- Como los valores pueden ser muy grandes, se usan prefijos:

| Prefijo | Símbolo | Equivale a | Ejemplo |
|---|---|---|---|
| — | Ω | 1 | 220 Ω |
| kilo | kΩ | 1 000 Ω | 4.7 kΩ = 4 700 Ω |
| mega | MΩ | 1 000 000 Ω | 1 MΩ |
| giga | GΩ | 1 000 000 000 Ω | 99 GΩ |

## 2. ¿Por qué tienen bandas de colores?

Las resistencias son muy pequeñas: un número impreso sería difícil de leer y se borraría. En cambio, **las bandas de colores se ven desde cualquier lado** y no se borran. Cada color representa un número según el **código de colores** (norma IEC 60062):

**Código de colores de resistencias (IEC 60062)**

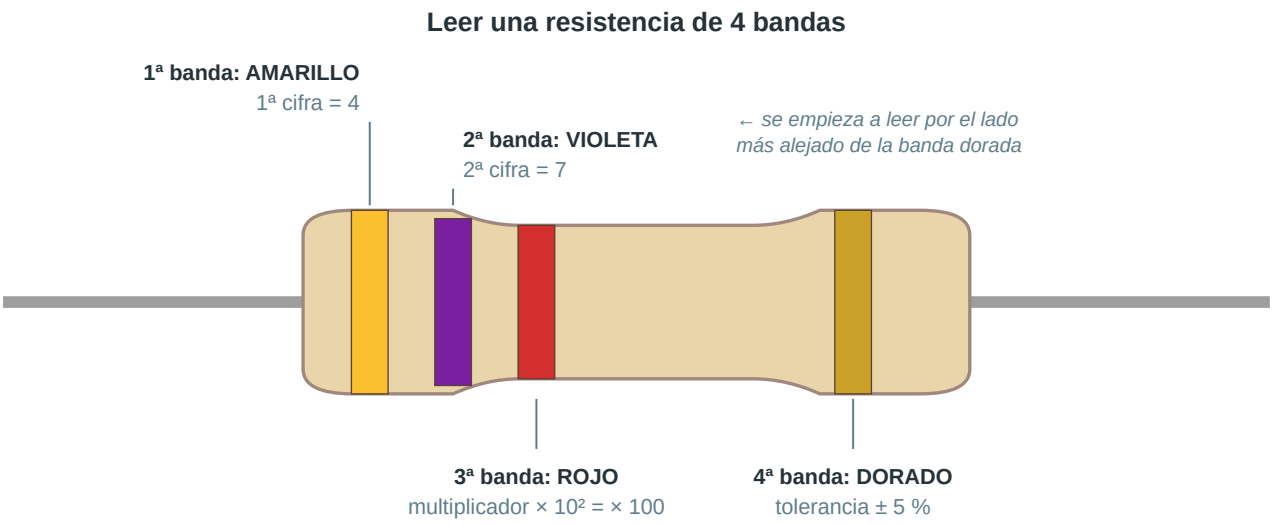
| Color | Cifra (bandas 1 y 2) | Multiplicador (banda 3) | | Tolerancia (banda 4) |
|---|---|---|---|---|
| **Negro** | 0 | × 1 | $10^0$ | — |
| **Marrón** | 1 | × 10 | $10^1$ | ± 1 % |
| **Rojo** | 2 | × 100 | $10^2$ | ± 2 % |
| **Naranja** | 3 | × 1 000 | $10^3$ | — |
| **Amarillo** | 4 | × 10 000 | $10^4$ | — |
| **Verde** | 5 | × 100 000 | $10^5$ | ± 0.5 % |
| **Azul** | 6 | × 1 000 000 | $10^6$ | ± 0.25 % |
| **Violeta** | 7 | × 10 000 000 | $10^7$ | ± 0.1 % |
| **Gris** | 8 | × 100 000 000 | $10^8$ | ± 0.05 % |
| **Blanco** | 9 | × 1 000 000 000 | $10^9$ | — |
| **Dorado** | – | × 0.1 | $10^{-1}$ | ± 5 % |
| **Plateado** | – | × 0.01 | $10^{-2}$ | ± 10 % |
| **Sin banda** | – | – | | ± 20 % |

Para recordar el orden del 0 al 9: «**N**i **M**e **R**obes **N**ada, **A**migo; **V**e **A**l **V**iejo **G**ran **B**anco»

Negro Marrón Rojo Naranja Amarillo Verde Azul Violeta Gris Blanco

*Tabla del código de colores*

# 3. Cómo leer una resistencia de 4 bandas




*Lectura de una resistencia amarillo-violeta-rojo-dorado*

1. Se empieza por el extremo **opuesto** a la banda dorada o plateada (la de tolerancia suele estar más separada).
2. **1.ª banda**: primera cifra. **2.ª banda**: segunda cifra. Juntas forman un número de dos cifras.
3. **3.ª banda**: el **multiplicador** (cuántos ceros se agregan, o sea, la potencia de 10).
4. **4.ª banda**: la **tolerancia** (cuánto puede variar el valor real).

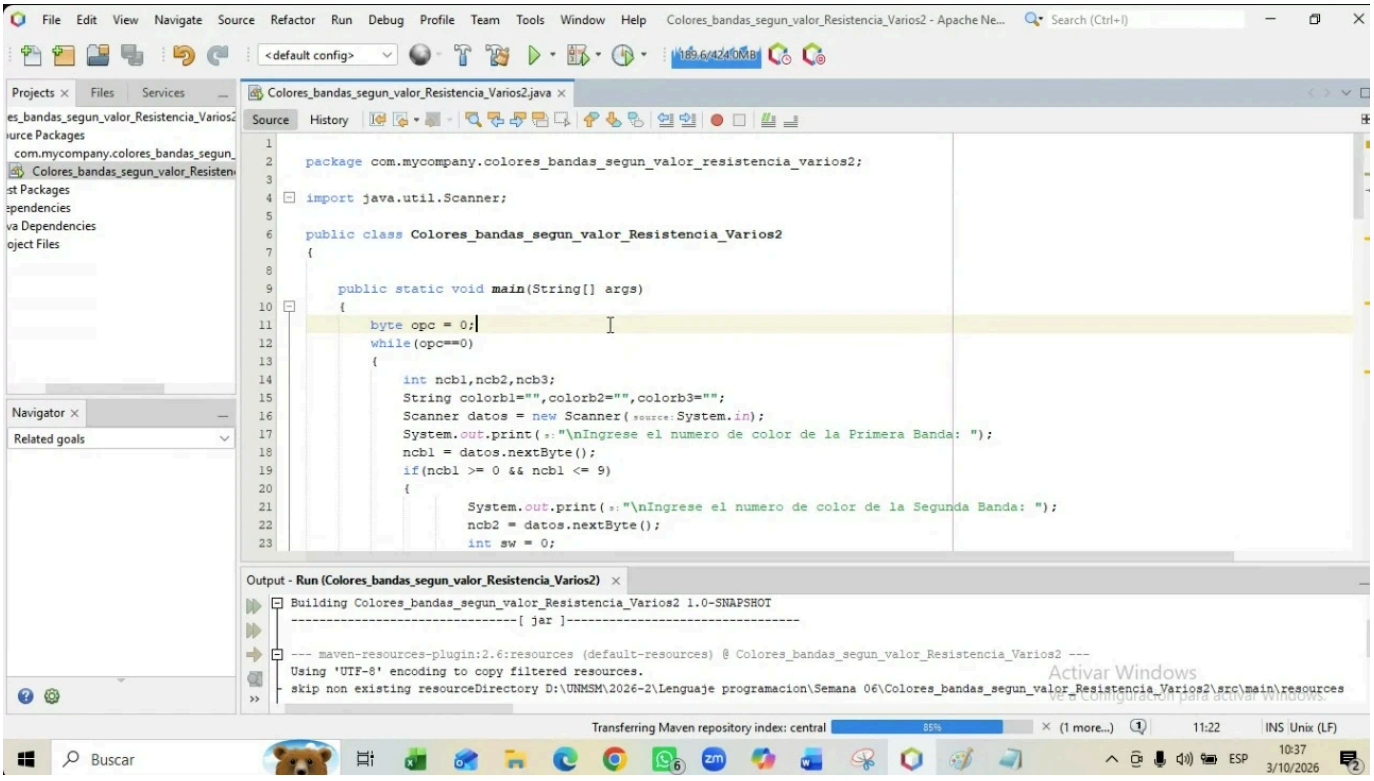
> **Fórmula:** R = (cifra$_1$ × 10 + cifra$_2$) × 10^(multiplicador)

| Colores | Cálculo | Valor |
|---|---|---|
| Marrón – Negro – Rojo | $10 \times 10^2$ | 1 000 Ω = **1 kΩ** |
| Rojo – Rojo – Negro | $22 \times 10^0$ | **22 Ω** |
| Amarillo – Violeta – Rojo | $47 \times 10^2$ | 4 700 Ω = **4.7 kΩ** |
| Verde – Azul – Naranja | $56 \times 10^3$ | 56 000 Ω = **56 kΩ** |
| Marrón – Negro – Verde | $10 \times 10^5$ | 1 000 000 Ω = **1 MΩ** |
| Marrón – Rojo – Dorado | $12 \times 10^{-1}$ | **1.2 Ω** |
| Naranja – Naranja – Plateado | $33 \times 10^{-2}$ | **0.33 Ω** |

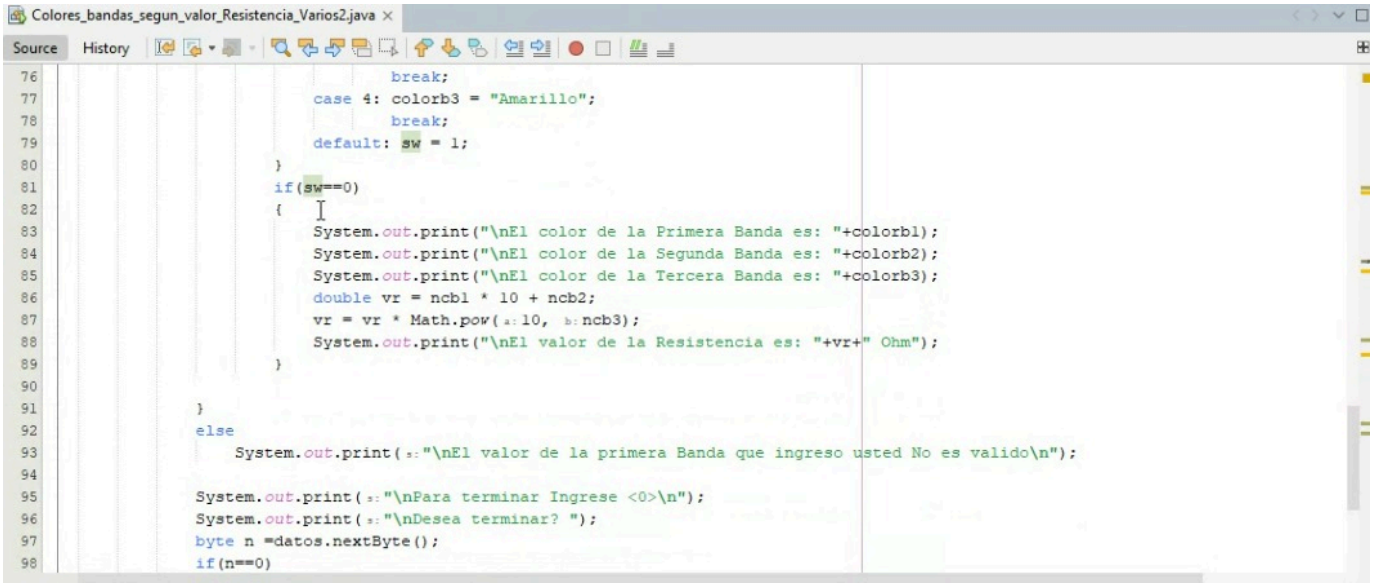
> **Nota**
>
> Algunas resistencias de precisión tienen **5 bandas** (tres cifras, multiplicador y tolerancia). Además, las tablas pueden variar en tolerancias poco usadas (la norma IEC 60062 asigna también ±0.05 % al naranja y ±0.02 % al amarillo). El programa del curso trabaja con las **tres primeras bandas**, que son las que dan el valor.

# 4. El programa de clase (el de las imágenes)

El programa del profesor, `Colores_bandas_segun_valor_Resistencia_Varios2`, hace el camino **número → color**: el usuario escribe el **número** de cada banda y el programa muestra **el nombre del color** y el valor de la resistencia.




*Inicio del programa en NetBeans (captura original)*




*Final del programa: muestra los colores y calcula el valor (captura original)*

Transcripción completa: `imagenes_originales/Colores_bandas_segun_valor_Resistencia_Varios2.java` (se conserva la misma numeración de líneas que en las capturas).

## Qué hace cada parte del programa de clase

| Líneas | Qué hace |
|---|---|
| 11–13 | `byte opc = 0;` y `while(opc==0)` : el programa se repite hasta que el usuario pida terminar |
| 14–16 | Declara los números de banda ( `ncb1` , `ncb2` , `ncb3` ), los nombres de color ( `colorb1` ...) y crea el `Scanner` |
| 17–18 | Pide el **número** de la 1.ª banda con `datos.nextByte()` |
| 19 | `if(ncb1 >= 0 && ncb1 <= 9)` : verifica que la 1.ª banda esté entre 0 y 9 |
| 21–23 | Pide la 2.ª banda y crea `sw = 0` (un «interruptor»: 0 = todo bien, 1 = hubo un error) |
| 25–42 | `switch(ncb1)` : según el número, guarda el nombre del color; si no coincide, `sw = 1` |
| 43–60 | Lo mismo para la 2.ª banda |
| 61–62 | Pide la 3.ª banda |
| 63–80 | Lo mismo para la 3.ª banda |
| 81–89 | Si `sw == 0` : muestra los tres colores y calcula `vr = (ncb1 * 10 + ncb2) * 10^ncb3` con `Math.pow` |
| 92–93 | Si la 1.ª banda no estaba entre 0 y 9, muestra un mensaje de error |
| 95–99 | Pregunta si desea terminar; si escriben 0, `opc = 1` y el ciclo termina |
| 101 | Muestra `****** FIN *******` |

Ejecución real de la transcripción (4-3-2 y luego 2-2-5):

```
Ingrese el numero de color de la Primera Banda: 4

Ingrese el numero de color de la Segunda Banda: 3

Ingrese el numero de color de la Tercera Banda: 2

El color de la Primera Banda es: Amarillo
El color de la Segunda Banda es: Naranja
El color de la Tercera Banda es: Rojo
El valor de la Resistencia es: 4300.0 Ohm
Para terminar Ingrese <0>

Desea terminar? 1

Ingrese el numero de color de la Primera Banda: 2

Ingrese el numero de color de la Segunda Banda: 2

Ingrese el numero de color de la Tercera Banda: 5

Para terminar Ingrese <0>

Desea terminar? 0

****** FIN *******
```

## Observaciones sobre el programa de clase

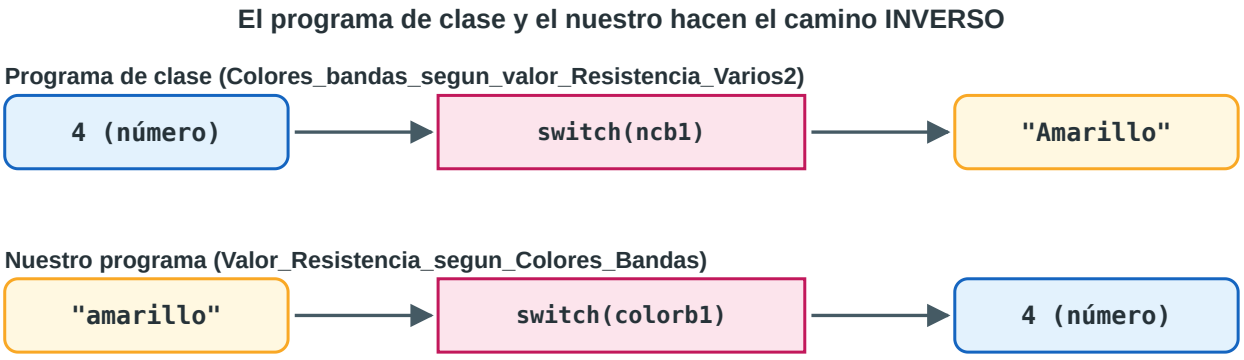
- Sus `switch` incluyen **7 de los 10 colores**: faltan **verde (5), azul (6) y violeta (7)**. Por eso, con 2-2-5 el programa **no muestra nada** (pone `sw = 1` sin avisar). Tampoco se puede calcular una resistencia tan común como amarillo-violeta-rojo (4.7 kΩ).
- No incluye el dorado ni el plateado del multiplicador.

- Si el usuario escribe una letra donde se espera un número, `nextByte()` detiene el programa con un error.
- El `Scanner` se crea en cada vuelta del ciclo (funciona, pero basta con crearlo una vez).

Nuestra versión mantiene **la misma estructura** (para que sea fácil compararlas) y agrega esos detalles.

## 5. Nuestro programa: colores → valor

La tarea pide hacer **el camino inverso**: el usuario escribe **los nombres de los colores** y el programa calcula **el número** (el valor de la resistencia).




Con los 3 números ya obtenidos, el cálculo es el mismo: vr = (ncb1 × 10 + ncb2) × 10^ncb3

*El programa de clase y el nuestro hacen el camino inverso*

### 5.1 Análisis Entrada – Proceso – Salida

| | Variable | Tipo | Descripción |
|---|---|---|---|
| **Entrada** | `colorb1`, `colorb2`, `colorb3` | `String` | Nombres de los colores de las 3 bandas (ej. `amarillo`) |
| **Entrada** | `n` | `String` | Respuesta a «¿Desea terminar?» (`0` = sí) |
| **Proceso** | `ncb1`, `ncb2` | `int` | Cifra de cada color (0 a 9), obtenida con `switch` |
| **Proceso** | `ncb3` | `int` | Potencia de 10 del multiplicador (−2 a 9) |
| **Proceso** | `sw` | `int` | 0 si todos los colores son válidos; 1 si alguno no lo es |
| **Proceso** | `vr` | `double` | `vr = (ncb1 × 10 + ncb2) × 10^ncb3` |
| **Salida** | — | — | La cifra de cada banda, el valor en Ohm y su equivalente en kOhm, MOhm o GOhm (o un mensaje de error) |

## 5.2 Pseudocódigo

```
Algoritmo ValorResistenciaSegunColores
    Definir colorb1, colorb2, colorb3, n Como Cadena
    Definir ncb1, ncb2, ncb3, sw, opc Como Entero
    Definir vr Como Real
    opc ← 0
    Mientras opc = 0 Hacer
        sw ← 0
        Leer colorb1, colorb2, colorb3          // en minúsculas
        Segun colorb1 Hacer
            "negro": ncb1 ← 0
            "marron": ncb1 ← 1
            ...                                 // rojo 2 ... blanco 9
            De Otro Modo: sw ← 1
                          Escribir "El color de la Primera Banda No es valido"
        FinSegun
        // lo mismo para colorb2 (ncb2)
        // y para colorb3 (ncb3), que además acepta dorado (-1) y plateado (-2)
        Si sw = 0 Entonces
            vr ← ncb1 * 10 + ncb2
            Si ncb3 >= 0 Entonces
                vr ← vr * 10 ^ ncb3
            SiNo
                vr ← vr / 10 ^ (-ncb3)
            FinSi
            Escribir "El valor de la Resistencia es: ", vr, " Ohm"
            Escribir el equivalente en kOhm, MOhm o GOhm
        SiNo
            Escribir "No se puede calcular: revise los colores."
        FinSi
        Escribir "Desea terminar? (0 = si)"
        Leer n
        Si n = "0" Entonces
            opc ← 1
        FinSi
    FinMientras
    Escribir "****** FIN *******"
FinAlgoritmo
```

## 5.3 Diagrama de flujo

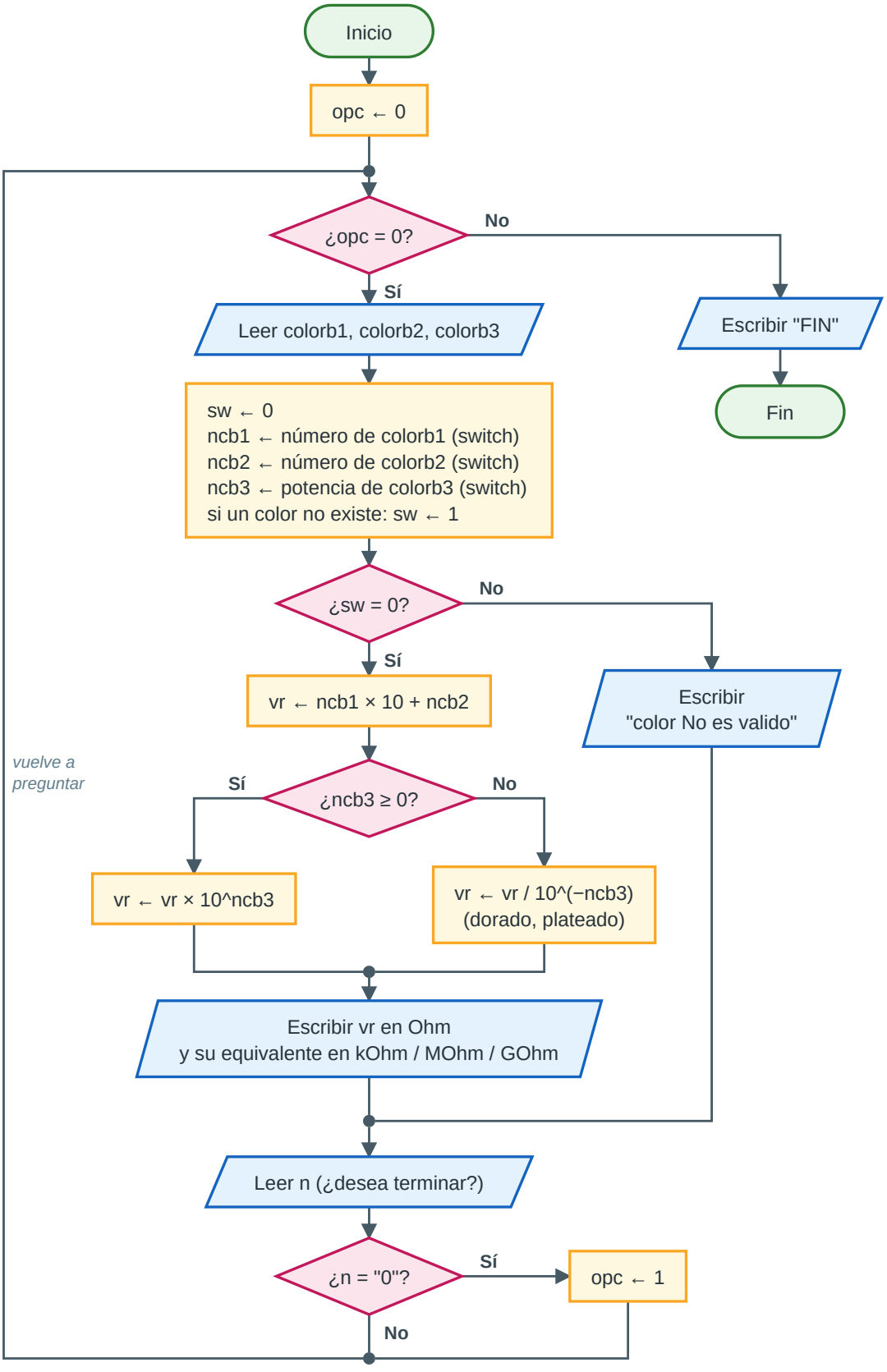



*Diagrama de flujo del programa de colores*

## 5.4 Código completo

Proyecto: `Parte2_Valor_Resistencia_segun_Colores/` · Archivo: `Valor_Resistencia_segun_Colores_Bandas.java`

```java
package com.mycompany.valor_resistencia_segun_colores_bandas;

import java.util.Scanner;

/*
 * VALOR DE UNA RESISTENCIA SEGUN LOS COLORES DE SUS BANDAS
 * --------------------------------------------------------
 * Es la version "al reves" del programa visto en clase
 * (Colores_bandas_segun_valor_Resistencia_Varios2):
 *   - En clase : se ingresaban NUMEROS y se mostraban los COLORES.
 *   - Aqui     : se ingresan los NOMBRES de los colores y el programa
 *                calcula el VALOR de la resistencia en Ohm.
 *
 *   Valor = (numero banda 1 x 10 + numero banda 2) x 10^(numero banda 3)
 *   Ejemplo: amarillo(4) violeta(7) rojo(2) -> 47 x 10^2 = 4700 Ohm
 */
public class Valor_Resistencia_segun_Colores_Bandas
{

    public static void main(String[] args)
    {
        Scanner datos = new Scanner(System.in);   // para leer lo que se escribe en el teclado
        byte opc = 0;                             // 0 = seguir, 1 = terminar

        System.out.print("===== VALOR DE UNA RESISTENCIA SEGUN SUS COLORES =====");
        System.out.print("\nColores validos (escribalos sin tildes):");
        System.out.print("\n  negro, marron, rojo, naranja, amarillo,");
        System.out.print("\n  verde, azul, violeta, gris, blanco");
        System.out.print("\n  (en la Tercera Banda tambien: dorado y plateado)\n");

        while(opc==0)
        {
            String colorb1, colorb2, colorb3;     // colores que escribe el usuario
            int ncb1 = 0, ncb2 = 0, ncb3 = 0;     // numero que le corresponde a cada color
            int sw = 0;                           // 0 = colores correctos, 1 = hubo un error

            // ---------- ENTRADA: se leen los NOMBRES de los colores ----------
            // toLowerCase() pasa todo a minusculas: "ROJO" y "Rojo" quedan como "rojo"
            System.out.print("\nIngrese el color de la Primera Banda: ");
            colorb1 = datos.next().toLowerCase();
            System.out.print("Ingrese el color de la Segunda Banda: ");
            colorb2 = datos.next().toLowerCase();
            System.out.print("Ingrese el color de la Tercera Banda: ");
            colorb3 = datos.next().toLowerCase();

            // ---------- PROCESO 1: convertir cada color en su numero ----------
            switch(colorb1)
            {
                case "negro":    ncb1 = 0;
                                 break;
                case "marron":   ncb1 = 1;
                                 break;
                case "rojo":     ncb1 = 2;
                                 break;
                case "naranja":  ncb1 = 3;
                                 break;
                case "amarillo": ncb1 = 4;
                                 break;
                case "verde":    ncb1 = 5;
                                 break;
                case "azul":     ncb1 = 6;
```

```
                        break;
            case "violeta":  ncb1 = 7;
                        break;
            case "gris":     ncb1 = 8;
                        break;
            case "blanco":   ncb1 = 9;
                        break;
            default: sw = 1;
                     System.out.print("\n * El color de la Primera Banda ("
                                 + colorb1 + ") No es valido");
        }
        switch(colorb2)
        {
            case "negro":    ncb2 = 0;
                        break;
            case "marron":   ncb2 = 1;
                        break;
            case "rojo":     ncb2 = 2;
                        break;
            case "naranja":  ncb2 = 3;
                        break;
            case "amarillo": ncb2 = 4;
                        break;
            case "verde":    ncb2 = 5;
                        break;
            case "azul":     ncb2 = 6;
                        break;
            case "violeta":  ncb2 = 7;
                        break;
            case "gris":     ncb2 = 8;
                        break;
            case "blanco":   ncb2 = 9;
                        break;
            default: sw = 1;
                     System.out.print("\n * El color de la Segunda Banda ("
                                 + colorb2 + ") No es valido");
        }
        // La Tercera Banda es el MULTIPLICADOR: ncb3 es la potencia de 10
        switch(colorb3)
        {
            case "negro":    ncb3 = 0;      // x 1
                        break;
            case "marron":   ncb3 = 1;      // x 10
                        break;
            case "rojo":     ncb3 = 2;      // x 100
                        break;
            case "naranja":  ncb3 = 3;      // x 1 000
                        break;
            case "amarillo": ncb3 = 4;      // x 10 000
                        break;
            case "verde":    ncb3 = 5;      // x 100 000
                        break;
            case "azul":     ncb3 = 6;      // x 1 000 000
                        break;
            case "violeta":  ncb3 = 7;      // x 10 000 000
                        break;
            case "gris":     ncb3 = 8;      // x 100 000 000
                        break;
            case "blanco":   ncb3 = 9;      // x 1 000 000 000
                        break;
            case "dorado":   ncb3 = -1;     // x 0.1
                        break;
            case "plateado": ncb3 = -2;     // x 0.01
                        break;
            default: sw = 1;
                     System.out.print("\n * El color de la Tercera Banda ("
                                 + colorb3 + ") No es valido");
        }
```

```
            // ---------- PROCESO 2 y SALIDA ----------
            if(sw==0)
            {
                System.out.print("\nEl numero de la Primera Banda (" + colorb1 + ") es: " + ncb1);
                System.out.print("\nEl numero de la Segunda Banda (" + colorb2 + ") es: " + ncb2);
                System.out.print("\nLa Tercera Banda (" + colorb3 + ") multiplica por: 10^"
                                 + ncb3);

                double vr = ncb1 * 10 + ncb2;          // 2 primeras bandas: 4 y 7 -> 47
                if(ncb3 >= 0)
                    vr = vr * Math.pow(10, ncb3);      // 47 x 10^2 = 4700
                else
                    vr = vr / Math.pow(10, -ncb3);     // dorado: 47 / 10 = 4.7 (ver nota 1)

                System.out.print("\nEl valor de la Resistencia es: " + vr + " Ohm");

                // Ademas lo mostramos en la unidad mas comoda (ver nota 2)
                if(vr >= 1000000000)
                    System.out.print("\nEquivale a: " + (vr / 1000000000) + " GOhm");
                else if(vr >= 1000000)
                    System.out.print("\nEquivale a: " + (vr / 1000000) + " MOhm");
                else if(vr >= 1000)
                    System.out.print("\nEquivale a: " + (vr / 1000) + " kOhm");
            }
            else
                System.out.print("\nNo se puede calcular: revise los colores.");

            System.out.print("\n\nPara terminar Ingrese <0>  (otro valor para continuar)");
            System.out.print("\nDesea terminar? ");
            String n = datos.next();          // se lee como texto: no falla si escriben una letra
            if(n.equals("0"))                 // los String se comparan con equals, NO con ==
                opc = 1;
        }
        System.out.print("\n****** FIN *******\n");
    }
}

/*
 * NOTA 1 - Por que DIVIDIMOS entre 10 en vez de multiplicar por 0.1:
 *   0.1 no se puede guardar exacto en binario (ver Parte 1: numeros reales
 *   en memoria). Por eso 12 * 0.1 da 1.2000000000000002, mientras que
 *   12 / 10 da 1.2 (el valor guardado mas cercano posible).
 *
 * NOTA 2 - Java muestra los double muy grandes en notacion cientifica:
 *   47000000.0 se imprime como 4.7E7 (es decir 4.7 x 10^7). Por eso
 *   tambien mostramos el valor en kOhm, MOhm o GOhm.
 */
```

## 5.5 Explicación por bloques

| Líneas | Código | Qué hace | Equivalente en C++ |
|---|---|---|---|
| 2–4 | `package …` / `import java.util.Scanner;` | Indica el paquete y trae la clase `Scanner` | `#include <iostream>` |
| 18–22 | `public class …` / `public static void main(…)` | La clase y el punto de inicio | `int main()` |
| 23 | `Scanner datos = new Scanner(System.in);` | Crea el lector del teclado (una sola vez) | `cin` ya existe |
| 24 | `byte opc = 0;` | Controla el ciclo: 0 = seguir | `int opc = 0;` |
| 26–30 | `System.out.print(…)` | Muestra la lista de colores válidos una vez | `cout << …` |
| 32 | `while(opc==0)` | Repite todo mientras `opc` valga 0 | igual |
| 34–36 | Variables | Los textos de color, sus números y `sw` | igual |
| 40–45 | `datos.next().toLowerCase()` | Lee **una palabra** y la pasa a minúsculas: `ROJO`, `Rojo` y `rojo` valen lo mismo | `cin >> colorb1;` |
| 48–73 | `switch(colorb1)` | Convierte el **nombre** del color en su **cifra** (0–9); si no existe, `sw = 1` y avisa | `if` / `else if` (C++ no permite `switch` con textos) |
| 74–99 | `switch(colorb2)` | Lo mismo para la 2.ª banda | |
| 101–130 | `switch(colorb3)` | Convierte el color en la **potencia de 10** del multiplicador; acepta dorado (−1) y plateado (−2) | |
| 133–138 | `if(sw==0)` y mensajes | Si todos los colores son válidos, muestra la cifra de cada banda | `if (sw == 0)` |
| 140 | `double vr = ncb1 * 10 + ncb2;` | Forma el número de dos cifras: 4 y 7 → 47 | igual |
| 141–144 | `Math.pow(10, ncb3)` | Multiplica por 10^ncb3, o **divide** si el exponente es negativo (dorado/plateado) | `pow(10, ncb3)` con `<cmath>` |
| 146 | `"… " + vr + " Ohm"` | Muestra el valor en ohmios | `cout << vr` |
| 149–154 | `if / else if` | Muestra el valor en GOhm, MOhm o kOhm según su tamaño | igual |
| 156–157 | `else` | Mensaje si algún color no era válido | igual |
| 159–163 | `datos.next()` y `n.equals("0")` | Pregunta si desea terminar; compara **textos con `equals`** | `if (n == "0")` |
| 165 | `****** FIN *******` | Mensaje final | |

## 5.6 Detalles importantes (las preguntas típicas)

**¿Por qué `toLowerCase()`?** Porque `switch` compara letra por letra y `"Rojo"` no es igual a `"rojo"`. Pasando todo a minúsculas, el usuario puede escribir como quiera.

**¿Por qué `switch` y no `if`?** Funcionarían ambos, pero cuando se compara **una misma variable contra muchos valores fijos**, el `switch` es más ordenado y fácil de leer. Además, es la estructura que usó el profesor. Desde Java 7, el `switch` acepta textos (`String`).

**¿Para qué sirve `sw`?** Es una **bandera** (*switch* o interruptor): empieza en 0 y, si algún `switch` llega a `default` (color inválido), se pone en 1. Al final, solo se calcula si `sw` sigue en 0. Es la misma técnica del profesor.

**¿Por qué `ncb3` puede ser negativo?** Porque el dorado multiplica por $0.1 = 10^{-1}$ y el plateado por $0.01 = 10^{-2}$. Guardar la **potencia** (−1, −2) permite usar la misma fórmula con `Math.pow`.

**¿Por qué dividir en lugar de multiplicar por 0.1?** Por lo visto en la Parte 1: 0.1 no es exacto en binario. Con la multiplicación, marrón-rojo-dorado daría `1.2000000000000002 Ohm`; con la división da `1.2 Ohm`. Se comprobó con las 1 200 combinaciones: multiplicando, 45 de ellas mostraban «ruido» de este tipo; dividiendo, ninguna.

**¿Qué es `4.7E8`?** Es la forma en que Java muestra los `double` grandes: $4.7 \times 10^8 = 470\,000\,000$. Por eso el programa también muestra el valor en MOhm (`470.0 MOhm`).

**¿Por qué la respuesta se lee como `String`?** Con `nextByte()` (como en el original), si el usuario escribe una letra el programa se cae. Leyendo texto, cualquier respuesta distinta de `0` simplemente continúa. Y los textos se comparan con `equals`, no con `==`.

## 5.7 Ejecución real

```
===== VALOR DE UNA RESISTENCIA SEGUN SUS COLORES =====
Colores validos (escribalos sin tildes):
  negro, marron, rojo, naranja, amarillo,
  verde, azul, violeta, gris, blanco
  (en la Tercera Banda tambien: dorado y plateado)

Ingrese el color de la Primera Banda: amarillo
Ingrese el color de la Segunda Banda: violeta
Ingrese el color de la Tercera Banda: rojo

El numero de la Primera Banda (amarillo) es: 4
El numero de la Segunda Banda (violeta) es: 7
La Tercera Banda (rojo) multiplica por: 10^2
El valor de la Resistencia es: 4700.0 Ohm
Equivale a: 4.7 kOhm

Para terminar Ingrese <0>  (otro valor para continuar)
Desea terminar? s

Ingrese el color de la Primera Banda: ROJO
Ingrese el color de la Segunda Banda: Rojo
Ingrese el color de la Tercera Banda: negro

El numero de la Primera Banda (rojo) es: 2
El numero de la Segunda Banda (rojo) es: 2
La Tercera Banda (negro) multiplica por: 10^0
El valor de la Resistencia es: 22.0 Ohm

Para terminar Ingrese <0>  (otro valor para continuar)
Desea terminar? s

Ingrese el color de la Primera Banda: marron
Ingrese el color de la Segunda Banda: rojo
Ingrese el color de la Tercera Banda: dorado

El numero de la Primera Banda (marron) es: 1
El numero de la Segunda Banda (rojo) es: 2
La Tercera Banda (dorado) multiplica por: 10^-1
El valor de la Resistencia es: 1.2 Ohm
```

```
Para terminar Ingrese <0>  (otro valor para continuar)
Desea terminar? s

Ingrese el color de la Primera Banda: amarillo
Ingrese el color de la Segunda Banda: violeta
Ingrese el color de la Tercera Banda: violeta

El numero de la Primera Banda (amarillo) es: 4
El numero de la Segunda Banda (violeta) es: 7
La Tercera Banda (violeta) multiplica por: 10^7
El valor de la Resistencia es: 4.7E8 Ohm
Equivale a: 470.0 MOhm

Para terminar Ingrese <0>  (otro valor para continuar)
Desea terminar? s

Ingrese el color de la Primera Banda: rosado
Ingrese el color de la Segunda Banda: negro
Ingrese el color de la Tercera Banda: rojo

 * El color de la Primera Banda (rosado) No es valido
No se puede calcular: revise los colores.

Para terminar Ingrese <0>  (otro valor para continuar)
Desea terminar? 0

****** FIN *******
```

## 5.8 Casos de prueba

| Colores ingresados | Cálculo | Esperado | Lo que muestra el programa |
|---|---|---|---|
| amarillo, violeta, rojo | $47 \times 10^2$ | 4.7 kΩ | `4700.0 Ohm` · `4.7 kOhm` ✔ |
| marron, negro, rojo | $10 \times 10^2$ | 1 kΩ | `1000.0 Ohm` · `1.0 kOhm` ✔ |
| ROJO, Rojo, negro | $22 \times 10^0$ | 22 Ω | `22.0 Ohm` ✔ (acepta mayúsculas) |
| marron, rojo, dorado | $12 \times 10^{-1}$ | 1.2 Ω | `1.2 Ohm` ✔ |
| naranja, naranja, plateado | $33 \times 10^{-2}$ | 0.33 Ω | `0.33 Ohm` ✔ |
| verde, azul, naranja | $56 \times 10^3$ | 56 kΩ | `56000.0 Ohm` · `56.0 kOhm` ✔ |
| marron, negro, verde | $10 \times 10^5$ | 1 MΩ | `1000000.0 Ohm` · `1.0 MOhm` ✔ |
| amarillo, violeta, violeta | $47 \times 10^7$ | 470 MΩ | `4.7E8 Ohm` · `470.0 MOhm` ✔ |
| blanco, blanco, blanco | $99 \times 10^9$ | 99 GΩ | `9.9E10 Ohm` · `99.0 GOhm` ✔ |
| rosado, negro, rojo | — | Error en la 1.ª banda | `El color de la Primera Banda (rosado) No es valido` ✔ |
| rojo, dorado, celeste | — | Error en la 2.ª y 3.ª banda | Dos mensajes de error ✔ (el dorado no es una cifra) |

### 5.9 Prueba de escritorio (amarillo, violeta, rojo)

| Paso | Instrucción | `colorb1` | `colorb2` | `colorb3` | `ncb1` | `ncb2` | `ncb3` | `sw` | `vr` |
|---|---|---|---|---|---|---|---|---|---|
| 1 | Leer los 3 colores | amarillo | violeta | rojo | 0 | 0 | 0 | 0 | |
| 2 | `switch(colorb1)` → `case "amarillo"` | | | | **4** | 0 | 0 | 0 | |
| 3 | `switch(colorb2)` → `case "violeta"` | | | | 4 | **7** | 0 | 0 | |
| 4 | `switch(colorb3)` → `case "rojo"` | | | | 4 | 7 | **2** | 0 | |
| 5 | `if(sw==0)` → verdadero | | | | | | | 0 | |
| 6 | `vr = ncb1 * 10 + ncb2` | | | | | | | | **47.0** |
| 7 | `ncb3 >= 0` → `vr = vr * Math.pow(10, 2)` | | | | | | | | **4700.0** |
| 8 | `vr >= 1000` → `vr / 1000` | | | | | | | | muestra **4.7 kOhm** |

## 6. Versión mejorada (opcional)

Archivo: `Valor_Resistencia_Version_Mejorada.java` (está en el mismo proyecto; se ejecuta con clic derecho → *Run File*).

Sirve si el profesor pregunta **«¿se puede mejorar?»**. Hace lo mismo, pero:

1. Usa **métodos**: el `switch` de colores se escribe **una sola vez** y se reutiliza para las tres bandas.
2. Lee la **4.ª banda (tolerancia)** y calcula el valor **mínimo** y **máximo** que puede tener la resistencia.
3. Muestra siempre el resultado en la unidad más cómoda y redondea el «ruido» de los `double`.

```java
// Devuelve el numero (0 a 9) que corresponde a un color, o -1 si no es valido.
static int numeroDelColor(String color)
{
    switch(color)
    {
        case "negro":    return 0;
        case "marron":   return 1;
        case "rojo":     return 2;
        case "naranja":  return 3;
        case "amarillo": return 4;
        case "verde":    return 5;
        case "azul":     return 6;
        case "violeta":  return 7;
        case "gris":     return 8;
        case "blanco":   return 9;
        default:         return -1;
    }
}
```

Ejecución real:

```
===== RESISTENCIA DE 4 BANDAS (version mejorada) =====

Banda 1 (1er digito)   : amarillo
Banda 2 (2do digito)   : violeta
Banda 3 (multiplicador): rojo
Banda 4 (tolerancia; si no tiene escriba ninguno): dorado

Valor nominal : 4.7 kOhm  +-5.0 %
Valor minimo  : 4.465 kOhm
Valor maximo  : 4.935 kOhm

Para terminar Ingrese <0>  (otro valor para continuar): s

Banda 1 (1er digito)   : marron
Banda 2 (2do digito)   : negro
Banda 3 (multiplicador): dorado
Banda 4 (tolerancia; si no tiene escriba ninguno): plateado

Valor nominal : 1.0 Ohm  +-10.0 %
Valor minimo  : 0.9 Ohm
Valor maximo  : 1.1 Ohm
```

# 7. Cómo ejecutar los programas en NetBeans

## Opción A — Abrir el proyecto incluido (la más rápida)

1. Copiar la carpeta `Parte2_Valor_Resistencia_segun_Colores` a la computadora.
2. En NetBeans: *File* → *Open Project…*, seleccionar esa carpeta (tiene el ícono de proyecto Maven) y *Open Project*.
3. Presionar `F6` (*Run Project*). La primera vez Maven puede tardar porque descarga lo necesario.
4. Escribir los colores en la ventana *Output* (abajo) y presionar Enter después de cada uno.
5. Para la versión mejorada: abrir `Valor_Resistencia_Version_Mejorada.java`, clic derecho → *Run File* (`Shift` + `F6`).

## Opción B — Crear el proyecto desde cero y pegar el código

1. *File* → *New Project…* → *Java with Maven* → *Java Application* → *Next*.
2. *Project Name*: `Valor_Resistencia_segun_Colores_Bandas` → *Finish*. NetBeans crea el paquete `com.mycompany.valor_resistencia_segun_colores_bandas` y una clase con el mismo nombre del proyecto.
3. Borrar el contenido de esa clase y pegar el código del punto 5.4.
4. Verificar que la línea `package …` del código coincida con el paquete que creó NetBeans (si el proyecto tiene otro nombre, cambiar solo esa línea y el nombre de la clase para que coincida con el nombre del archivo).
5. `F6` para ejecutar.

## Problemas frecuentes

| Problema | Causa | Solución |
|---|---|---|
| `class … is public, should be declared in a file named …` | El nombre de la clase no coincide con el del archivo | Usar el mismo nombre en ambos |
| `package … does not exist` o la clase no aparece | La línea `package` no coincide con la carpeta | Corregir la línea `package` |
| Escribo `Marrón` y dice que no es válido | Tilde | Escribir sin tildes: `marron` |
| No pasa nada al escribir | Se está escribiendo en el editor y no en *Output* | Hacer clic en la ventana *Output* antes de escribir |

# Guion de exposición

## 1. Objetivo y público

**Objetivo:** que al terminar, los compañeros puedan responder dos preguntas:

1. ¿Cómo guarda la computadora un número con decimales (y por qué a veces «se equivoca»)?
2. ¿Cómo funciona un programa que, a partir de los **colores** de una resistencia, calcula su **valor**?

**Público:** compañeros que vieron algo de C++ pero **no saben Java**. Por eso, la estrategia de toda la exposición es:

> **Primero la idea (con un dibujo), después el código, y siempre comparando con C++.**

## 2. Estructura y tiempos

Pensado para **15 a 20 minutos** y un grupo de **3 personas** (A, B y C). Si son 2, A hace los bloques 1–3 y B los bloques 4–8; si son 4, el cuarto integrante hace las dos demostraciones.

| N.° | Bloque | Tiempo | Quién | Material |
|---|---|---|---|---|
| 1 | Apertura y pregunta gancho | 1 min | A | Diapositiva «¿0.1 + 0.2 = 0.3?» |
| 2 | Parte 1: cómo se guarda un número real | 5 min | A | Figuras de bits, binario, IEEE 754 y ejemplos |
| 3 | Demostración 1 (programa de números reales) | 2 min | A o C | NetBeans: `-6.75`, `0.1` y curiosidades |
| 4 | Parte 2: resistencias y código de colores | 2 min | B | Tabla de colores y dibujo de la resistencia |
| 5 | Parte 2: el programa de clase y el nuestro | 3–4 min | B | Comparación, tabla C++ ↔ Java, código clave, diagrama de flujo |
| 6 | Demostración 2 (programa de colores) | 2 min | C | NetBeans: lista de casos de prueba |
| 7 | Conexión entre las dos partes | 1 min | C | El ejemplo 12 × 0.1 |
| 8 | Conclusiones y preguntas | 2–3 min | Todos | Resumen en 3 ideas |

> **Consejo**
>
> Si el profesor da menos tiempo, recorta los bloques 2 y 5 (no las demostraciones): una demo que funciona convence más que cualquier diapositiva.

## 3. Antes del día de la exposición

- ☐ **Probar los dos programas en la computadora donde se va a exponer.** Si es otra PC, abrir los proyectos un día antes: la primera vez Maven necesita internet para descargar sus componentes.
- ☐ **Agrandar la letra** del editor y de la ventana *Output* de NetBeans (*Tools → Options → Fonts & Colors* y *Tools → Options → Miscellaneous → Output*) para que se lea desde el fondo del salón.
- ☐ Tener **impresa la lista de datos de prueba** (sección 7) para no improvisar.
- ☐ Tener un **plan B**: este PDF abierto en las ejecuciones reales (o capturas de pantalla) por si NetBeans falla.

- [ ] Cerrar programas y notificaciones; poner la PC en modo «no molestar».
- [ ] **Ensayar dos veces con cronómetro**, cada integrante con su parte.
- [ ] Repartir las preguntas frecuentes (sección 9): cada uno debe dominar las de su parte.

## 4. Diapositivas sugeridas

Todas las figuras están en la carpeta `diagramas/` (archivos `.svg`, se pueden insertar en PowerPoint o Google Slides).

| N.° | Título | Contenido | Figura |
|---|---|---|---|
| 1 | Portada | Título, curso, integrantes | — |
| 2 | ¿0.1 + 0.2 = 0.3? | La pregunta gancho y la respuesta de la computadora | — |
| 3 | Bits, bytes y memoria | Bit, byte, direcciones; `float` = 4 bytes, `double` = 8 | `memoria_bytes.svg` |
| 4 | De decimal a binario | Divisiones (parte entera) y multiplicaciones (parte fraccionaria) | `conversion_binario.svg` |
| 5 | Punto flotante | Notación científica en base 2 | `notacion_cientifica.svg` |
| 6 | Estándar IEEE 754 | Signo, exponente (con sesgo) y mantisa en `float` y `double` | `ieee754_formatos.svg` |
| 7 | Ejemplo: −6.75 | Los 32 bits y su lectura: C0D80000 | `ieee754_ejemplo_menos_6_75.svg` |
| 8 | Ejemplo: 0.1 | Binario infinito, redondeo y valor guardado | `ieee754_ejemplo_0_1.svg` |
| 9 | Demostración 1 | Captura del programa (por si se necesita) | — |
| 10 | La resistencia y su código de colores | Qué es, por qué colores, tabla y regla para recordar | `tabla_colores.svg`, `resistencia_bandas.svg` |
| 11 | Programa de clase vs. el nuestro | Número → color vs. color → número | `comparacion_programas.svg` |
| 12 | Java en 1 minuto (para quien sabe C++) | Tabla de equivalencias reducida | — |
| 13 | El código clave | `switch` con texto, `toLowerCase`, `Math.pow`, dividir en vez de multiplicar | — |
| 14 | Diagrama de flujo | El programa completo de un vistazo | `flujo_resistencia.svg` |
| 15 | Demostración 2 | Casos de prueba | — |
| 16 | Conclusiones | Las 3 ideas | — |
| 17 | ¿Preguntas? | — | — |

Tabla reducida para la diapositiva 12:

| C++ | Java |
|---|---|
| `cout << "Hola";` | `System.out.print("Hola");` |
| `cin >> color;` | `color = datos.next();` |
| `string` | `String` |
| `pow(10, 2)` | `Math.pow(10, 2)` |
| `if`, `switch`, `while` | ¡Iguales! |

## 5. Guion: qué decir en cada bloque

Las frases entre comillas son una guía; no hay que memorizarlas, pero sí la **idea** de cada una.

### Bloque 1 — Apertura (A, 1 min)

> «Buenos días. Empezamos con una pregunta: si le pedimos a la computadora que sume 0.1 más 0.2, ¿qué creen que responde? … Responde **0.30000000000000004**. No es un error de Java: pasa en casi todos los lenguajes. Hoy vamos a explicar **por qué**, es decir, **cómo se guarda un número real en la memoria**. Y después les mostraremos un programa en Java que calcula **el valor de una resistencia a partir de sus colores**, y verán que las dos partes están conectadas.»

### Bloque 2 — Parte 1: el número real en memoria (A, 5 min)

**Diapositiva 3 (bits y bytes):**

> «La computadora solo guarda ceros y unos: los **bits**. Ocho bits forman un **byte**. La memoria RAM es como una fila enorme de casilleros de un byte, y cada casillero tiene un número, su dirección. Un `float` ocupa 4 casilleros y un `double`, 8.»

**Diapositiva 4 (binario):**

> «Para guardar 6.75, primero lo pasamos a binario. La parte entera, 6, se divide entre 2 y se leen los restos de abajo hacia arriba: 110. La parte decimal, 0.75, se multiplica por 2 y se toman las partes enteras: 0.11. Entonces 6.75 es **110.11** en binario.»

> «Pero con 0.1 pasa algo: al multiplicar por 2 nunca llegamos a cero; el patrón 0011 se repite **para siempre**. Es como 1/3 en decimal, 0.333… infinito. La memoria es finita, así que **hay que cortar y redondear**. Ahí nace el error.»

**Diapositiva 5 (punto flotante):**

> «Para guardar números muy grandes y muy pequeños se usa la **notación científica**, como en física: 6 750 000 es 6.75 por 10 a la 6. La computadora hace lo mismo pero en base 2: 110.11 es **1.1011 por 2 al cuadrado**. Como el punto se mueve, se llama **punto flotante**: de ahí viene el nombre `float`.»

**Diapositiva 6 (IEEE 754):**

> «Todas las computadoras siguen el estándar **IEEE 754**. Un `float` tiene 32 bits: **1 bit de signo**, **8 de exponente** y **23 de mantisa**. Dos trucos: el exponente se guarda **sumándole 127** (el *sesgo*), para poder tener exponentes negativos; y el «1.» del inicio **no se guarda**, porque en binario siempre es 1. El `double` es igual pero con 64 bits: 1, 11 y 52, y sesgo 1023.»

**Diapositiva 7 (−6.75):**

> «Armemos −6.75: es negativo, así que el signo es 1. Exponente: 2 más 127 es 129, en binario 10000001. Mantisa: lo que va después del «1.», o sea 1011 y el resto ceros. Juntando todo, en hexadecimal: **C0D80000**. Y si lo leemos al revés con la fórmula, da exactamente −6.75.»

**Diapositiva 8 (0.1):**

> «Con 0.1, la mantisa tiene que cortarse en 23 bits y el último se redondea. Por eso, lo que realmente queda guardado es 0.100000001490116… **No es exactamente 0.1.** Cuando sumamos 0.1 y 0.2, esos pequeños errores se juntan y aparece el 0.30000000000000004. Conclusión práctica: **los reales no se comparan con `==`**, se compara con una tolerancia.»

## Bloque 3 — Demostración 1 (2 min)

1. Abrir `Numero_Real_en_Memoria` y presionar `F6`.
2. Escribir `1` y luego `-6.75`.

> «Miren: el programa hace la misma conversión que hicimos en la pizarra, paso a paso: las divisiones, las multiplicaciones, el exponente con sesgo y la mantisa. Y abajo compara con los bits que **Java realmente guarda**: **coinciden**. También muestra el hexadecimal C0D80000 y cómo quedan los 4 bytes en la memoria.»

3. Escribir `1` y luego `0.1`.

> «Ahora 0.1: la fracción nunca llega a cero, el último bit se redondea, y el valor exacto guardado no es 0.1.»

4. Escribir `2` (curiosidades).

> «Y aquí el 0.1 + 0.2, y el detalle que conecta con la segunda parte: 12 por 0.1 da 1.2000000000000002.»

## Bloque 4 — Resistencias y código de colores (B, 2 min)

> «Una **resistencia** es un componente que se opone al paso de la corriente; se mide en **ohmios**. Como son tan pequeñas, en vez de números se pintan **bandas de colores**. Cada color es un número del 0 al 9. Para recordar el orden: ***Ni Me Robes Nada, Amigo; Ve Al Viejo Gran Banco***: Negro, Marrón, Rojo, Naranja, Amarillo, Verde, Azul, Violeta, Gris, Blanco.»

> «Se lee así: la 1.ª y 2.ª banda son las cifras; la 3.ª dice por cuánto multiplicar; la 4.ª es la tolerancia. Amarillo-violeta-rojo es 4, 7 y por 100: **4 700 ohmios, o 4.7 kilo-ohmios**.»

## Bloque 5 — El programa de clase y el nuestro (B, 3–4 min)

**Diapositiva 11:**

> «En clase vimos un programa que recibe **números** y muestra los **colores**. La tarea era hacer lo contrario: que el usuario escriba **los nombres de los colores** y el programa calcule **el valor**. Mantuvimos la misma estructura del profesor para que se pueda comparar.»

**Diapositiva 12 (Java para quien sabe C++):**

> «Si no conocen Java, no se preocupen: es muy parecido a C++. `System.out.print` es como `cout`; `Scanner` es como `cin`; `String` es `string`. Y `if`, `switch` y `while` funcionan **igual**. Una advertencia: en las capturas de NetBeans aparecen cosas como «s:» o «a:» en gris; eso **no es código**, es una ayuda que dibuja NetBeans con el nombre de los parámetros.»

**Diapositiva 13 (el código clave)** — mostrar solo estos fragmentos, con letra grande:

```
colorb1 = datos.next().toLowerCase();   // lee "Amarillo" y lo deja "amarillo"

switch(colorb1)
{
    case "amarillo": ncb1 = 4;
                     break;
    // ... un case por cada color
    default: sw = 1;                     // color que no existe
}

double vr = ncb1 * 10 + ncb2;           // 4 y 7 -> 47
vr = vr * Math.pow(10, ncb3);           // 47 x 10^2 = 4700
```

> «Primero leemos cada color y lo pasamos a minúsculas, así da igual si escriben ROJO o Rojo. Después, con un `switch` convertimos cada nombre en su número; en Java el `switch` puede comparar textos. Si alguien escribe un color que no existe, entra al `default` y marcamos un error con la variable `sw`. Al final, la fórmula: la primera cifra por 10 más la segunda, por 10 elevado al multiplicador, con `Math.pow`.»

**Diapositiva 14 (diagrama de flujo):**

> «Este es el programa completo: un ciclo que pide los 3 colores, los convierte, verifica que sean válidos, calcula y pregunta si desea terminar.»

## Bloque 6 — Demostración 2 (C, 2 min)

Abrir `Valor_Resistencia_segun_Colores_Bandas`, `F6`, e ingresar en este orden:

| Ingresar | Debe mostrar | Qué comentar |
|---|---|---|
| `amarillo`, `violeta`, `rojo` | `4700.0 Ohm` · `4.7 kOhm` | El ejemplo de la diapositiva |
| `ROJO`, `Rojo`, `negro` | `22.0 Ohm` | Acepta mayúsculas |
| `marron`, `rojo`, `dorado` | `1.2 Ohm` | El dorado multiplica por 0.1 |
| `amarillo`, `violeta`, `violeta` | `4.7E8 Ohm` · `470.0 MOhm` | `E8` es notación científica: × $10^8$ |
| `rosado`, `negro`, `rojo` | Mensaje de color no válido | Validación con `default` |
| `0` | `****** FIN *******` | Termina el ciclo |

### Bloque 7 — Conexión entre las partes (C, 1 min)

> «¿Recuerdan el 0.1 que no se puede guardar exacto? Con la banda dorada hay que multiplicar por 0.1, y 12 por 0.1 da **1.2000000000000002**. Por eso, en nuestro programa, en lugar de multiplicar por 0.1 **dividimos entre 10**, y sale 1.2 limpio. Lo comprobamos con las 1 200 combinaciones posibles de colores: todas dan el valor correcto.»

### Bloque 8 — Conclusiones (todos, 1 min + preguntas)

> «Tres ideas para llevarse:
>
> 1. Los números reales se guardan como **signo, exponente y mantisa** (IEEE 754), y muchos decimales como 0.1 **no son exactos** en binario.
> 2. Por eso los reales no se comparan con `==` y a veces conviene dividir en lugar de multiplicar por un decimal.
> 3. Con un `switch` convertimos los **nombres de los colores** en números y calculamos el valor de la resistencia con su fórmula. Muchas gracias. ¿Preguntas?»

## 6. Cómo explicar Java a quien no sabe Java

1. **Compara siempre con C++**: «esto es como `cout`», «esto es como `cin`».
2. **Diagrama antes que código**: primero el diagrama de flujo o la figura; después el fragmento de código.
3. **No leas el programa completo**: muestra solo las líneas importantes (lectura, `switch`, fórmula), con la letra bien grande.
4. **Señala con el cursor** la línea de la que hablas y di su número.
5. **Evita palabras técnicas** innecesarias (JVM, bytecode, Maven). Si alguien pregunta, una frase basta: «Java se traduce a un código intermedio que funciona en cualquier sistema».
6. **Explica lo gris de NetBeans** (`s:`, `a:`) al inicio, para que nadie se confunda.
7. **Usa ejemplos concretos**: «amarillo es 4, violeta es 7», no «la variable `ncb1` toma el valor del dígito».

## 7. Datos de prueba (para tener impresos)

**Programa 1 (números reales):** `1` → `-6.75` · `1` → `0.1` · `1` → `4700` · `2` (curiosidades) · `0` (salir).

**Programa 2 (resistencias):** amarillo violeta rojo · ROJO Rojo negro · marron rojo dorado · amarillo violeta violeta · rosado negro rojo · `0`.

## 8. Errores que conviene evitar al exponer

- Leer las diapositivas o el código palabra por palabra.
- Poner mucho texto en una diapositiva (máximo 5 líneas o una figura).
- Hacer la demostración sin haberla probado en esa computadora.
- Usar números o colores improvisados en la demo (usa la lista de la sección 7).
- Hablar mirando la pantalla en vez de al público.
- Pasarse del tiempo: ensaya con cronómetro.

## 9. Preguntas frecuentes (y cómo responderlas)

| Pregunta | Respuesta corta |
|---|---|
| ¿Por qué usan `double` y no `float`? | `double` tiene el doble de bits y unas 16 cifras de precisión (el `float`, unas 7). Es el tipo por defecto para decimales en Java y C++. |
| ¿Qué pasa si escribo el color con mayúsculas? | Funciona: `toLowerCase()` lo convierte a minúsculas antes del `switch`. |
| ¿Y si escribo «marrón» con tilde? | Hay que escribirlo sin tilde (`marron`), como en el programa del profesor; con tilde aparece como color no válido. |
| ¿Por qué `switch` y no `if`? | Ambos sirven; el `switch` es más claro para comparar una variable con muchos valores fijos, y es lo que usó el profesor. Java permite `switch` con textos desde la versión 7. |
| ¿Qué hace `Math.pow(10, ncb3)`? | Calcula 10 elevado a `ncb3`: $10^2$, $10^3$… Es el multiplicador. |
| ¿Por qué sale `4.7E8`? | Es notación científica de Java: $4.7 \times 10^8$. Por eso también mostramos `470.0 MOhm`. |
| ¿Por qué dividen entre 10 en vez de multiplicar por 0.1? | Porque 0.1 no es exacto en binario: 12 × 0.1 = 1.2000000000000002, mientras que 12 / 10 = 1.2. |
| ¿Qué es la variable `sw`? | Una «bandera»: empieza en 0 y se pone en 1 si algún color no es válido; solo se calcula si sigue en 0. |
| ¿Qué es el sesgo? | Un número que se suma al exponente (127 en `float`) para guardar exponentes negativos sin usar signo. |
| ¿Qué es el bit implícito? | El «1.» inicial de la mantisa: siempre es 1, por eso no se guarda y se gana un bit de precisión. |
| ¿Por qué 0.1 + 0.2 no da 0.3? | Ni 0.1 ni 0.2 se guardan exactos (son infinitos en binario); sus pequeños errores se suman. |
| ¿Qué es little-endian? | Que la PC guarda primero el byte menos significativo: −6.75f (C0 D8 00 00) queda en memoria como 00 00 D8 C0. |
| ¿Por qué el programa de clase no muestra nada con 5, 6 o 7? | Sus `switch` no tienen los casos verde, azul y violeta; caen en `default` y no se muestra ningún mensaje. Nuestro programa incluye los 10 colores y avisa los errores. |
| ¿Se puede considerar la tolerancia? | Sí: la versión mejorada lee la 4.ª banda y muestra el valor mínimo y máximo. |
| ¿Qué pasa si escribo una letra en «Desea terminar»? | Nada malo: se lee como texto; cualquier respuesta distinta de `0` continúa. |
| ¿Qué significa `public static void main(String[] args)`? | Es el punto de inicio del programa, como `int main()` en C++. |
| ¿Qué son «s:» y «a:» en NetBeans? | Ayudas visuales con el nombre de los parámetros; no forman parte del código. |

## 10. Plan B si la demostración falla

1. Mantener la calma: «Mientras se soluciona, les muestro la ejecución que hicimos antes».
2. Abrir este PDF en la sección de **ejecución real** de cada programa (Parte 1: sección 13.2; Parte 2: sección 5.7) o mostrar capturas tomadas el día anterior.
3. Explicar con la **prueba de escritorio** (Parte 2, sección 5.9): se puede seguir a mano en la pizarra.

# 11. Tarjeta de bolsillo

| Dato | Valor |
|---|---|
| `float` | 32 bits: signo 1 · exponente 8 (sesgo 127) · mantisa 23 · ~7 cifras |
| `double` | 64 bits: signo 1 · exponente 11 (sesgo 1023) · mantisa 52 · ~16 cifras |
| Fórmula | valor = (−1)^s × 1.mantisa × 2^(exponente − sesgo) |
| −6.75 | $110.11_2 = 1.1011 \times 2^2 \rightarrow$ `C0D80000` (exacto) |
| 0.1 | $0.000110011\ldots_2$ (infinito) → `3DCCCCCD` → 0.100000001490116… |
| Colores 0–9 | Negro, Marrón, Rojo, Naranja, Amarillo, Verde, Azul, Violeta, Gris, Blanco |
| Fórmula resistencia | $(\text{cifra}_1 \times 10 + \text{cifra}_2) \times 10^{\text{multiplicador}}$ |
| Ejemplo | amarillo-violeta-rojo = 47 × 100 = 4 700 Ω = 4.7 kΩ |
| NetBeans | `F6` ejecutar proyecto · `Shift`+`F6` ejecutar archivo |